UNIVERSITI POLY-TECH MALAYSIA

ESP32-Based IoT Honeypot for Intrusion Detection and Forensic Logging

SYED AMER AIDID BIN
SYED MOHD BAKRI

BACHELOR OF INFORMATION TECHNOLOGY (HONS) IN CYBER SECURITY

UNIVERSITI POLY-TECH MALAYSIA
Faculty of Computing & Multimedia

ESP32-Based IoT Honeypot for Intrusion Detection and Forensic Logging

SYED AMER AIDID BIN SYED MOHD BAKRI
AM2412018308

FYP4112

MAY 2026

Declaration of Originality

This project is all my own work and has not been copied in part or in whole from any other source except where duly acknowledged. As such, all use of previously published work (from books, journals, magazines, internet, etc.) has been acknowledged within the main report to an item in the References or Bibliography lists.

I also agree that an electronic copy of this project may be stored and used for the purposes of plagiarism prevention and detection.

Copyright Acknowledgement

I acknowledge that the copyright of this project and report belongs to Universiti Poly-Tech Malaysia.

Signed:  Date: 26.6.2026

Office Stamp

Abstract

Small and medium-sized enterprises (SMEs) increasingly rely on internet-connected devices to support daily business operations, yet many operate their networks without any mechanism to monitor for unauthorised access attempts. This project addresses this gap by proposing an ESP32-Based IoT Honeypot System for Intrusion Detection and Security Monitoring, developed for Yumplingy Enterprise, a small dessert and food business that recently installed a Wi-Fi network to support its operations but currently has no form of network security monitoring in place. Three problems were identified to motivate this project: the absence of any network monitoring mechanism, the growing prevalence of automated attacks targeting small business networks, and the lack of any means to capture and analyse attacker behaviour. In response, three objectives were established: to develop a honeypot device using an ESP32 microcontroller that simulates a vulnerable IoT device by advertising fake Telnet and HTTP administration panel services; to detect and classify common attack techniques, including port scanning, brute-force login attempts, and unauthorised connection attempts, in real time; and to log all detected attack evidence into a structured MySQL database and display it through a Django web dashboard with alerts and basic forensic reporting.

This report covers the work completed during the first phase of the project. A literature review and comparison of three existing honeypot frameworks, namely Honeyd, Cowrie, and T-Pot, revealed that while each offers an effective detection mechanism, all three require server-class infrastructure and technical expertise that a small business such as Yumplingy Enterprise does not possess, confirming a clear research gap for a low-cost, microcontroller-based alternative. Requirements were then gathered through a questionnaire distributed to 50 respondents across various small business sectors and a structured interview conducted with Yumplingy Enterprise. The findings confirmed that 72% of small business respondents have no network monitoring system in place, that concern regarding cyberattacks (4.06 out of 5) far exceeds respondents' confidence in their own security posture (2.04 out of 5), and that cost and complexity of setup are the primary barriers preventing small businesses from adopting existing security solutions. These findings, together with a use case model, process flowchart, and Business Process Modelling Notation (BPMN) diagram developed to model the proposed system's behaviour, establish the foundation upon which the system will be designed, developed, and evaluated in the next phase of this project.

Table of Contents

1 INTRODUCTION 12
1.1 Introduction 12
1.2 Project Background 12
1.3 Problem Statement 14
1.3.1 Lack of network security monitoring 14
1.3.2 Growing cyber threats targeting office networks 14
1.3.3 No mechanism to capture and analyse attack behaviour 14
1.4 Project Objectives 16
1.4.1 To develop an ESP32-based honeypot device that simulates a vulnerable IoT device and monitors the office network for unauthorised access attempts. 16
1.4.2 To detect and classify common cyber attack techniques, including port scanning, brute-force login attempts, and unauthorised connection attempts, in real time. 16
1.4.3 To log detected attack evidence into a structured database and display it through a web dashboard with alerts and forensic reporting. 16
1.5 Scope and Target User 17
1.5.1 Project Scope 17
1.5.2 Product Scope 17
1.5.3 Target User 18
1.6 Conclusion 18

2 LITERATURE REVIEW 19
2.1 Introduction 19
2.2 Investigation 19
2.2.1 Definition and Mechanism of Honeypot-Based Intrusion Detection 21
2.2.2 Types of Honeypots by Interaction Level 21
2.2.3 Common Attack Techniques Targeting Small Office and IoT Networks 22
2.2.4 Case Studies of Low-Cost and Embedded Honeypot Deployments 22
2.2.5 Challenges and Limitations of Honeypot-Based Detection 23
2.3 Related Works 26
2.3.1 Honeyd 26
2.3.2 Cowrie SSH/Telnet Honeypot 28
2.3.3 T-Pot (All-in-One Honeypot Platform) 30

3 METHODOLOGY 33
3.1 Introduction 33
3.2 Agile Methodology 34
3.3 Phases in Agile Methodology 35
3.3.1 Phase 1: Plan 35
3.3.2 Phase 2: Design 35
3.3.3 Phase 3: Develop 35
3.3.4 Phase 4: Test 35
3.4 Requirement 37
3.4.1 Data Gathering Techniques 37
3.4.1.1 Questionnaire 37

3.4.1.2 Interview 37
3.4.2 Functional Requirements 38
3.4.3 Non-functional requirements 39
3.4.4 System Requirements 40
3.4.4.1 Hardware Requirements 40
3.4.4.2 Software Requirements 40
3.5 Analysis 41
3.5.1 Data Gathering Analysis 41
3.5.1.1 Questionnaire Analysis 41
3.5.1.2 Interview Analysis 51
3.5.2 Use Case Model 53
3.5.3 Flowchart 54
3.6 Conclusion 56

List of Figures

Figure 2.3.1.1 Illustrative example of Honeyd configuration commands and a connection log entry, constructed for this report based on the documented usage described in Provos (2004). 26
Figure 2.3.1.2 Honeyd configuration by (Gajjar, 2013) 26
Figure 2.3.1.3 Running the honeyd service (Gajjar, 2013) 26
Figure 2.3.2.1 Illustrative example of a Cowrie login attempt and emulated shell session, constructed for this report based on the deployment behaviour described in Zielinski and Kholidy (2023). 28
Figure 2.3.2.2 Configuration of cowrie (French, 2018) 28
Figure 2.3.2.3 Cowrie Log (French, 2018) 28
Figure 2.3.3.1 Illustrative example of a T-Pot-style dashboard summary view, constructed for this report based on the dashboard capabilities described in Telekom Security (2024). 30
Figure 2.3.3.2 Need to have VPS to host it (Bellekens, 2022) 30
Figure 2.3.3.3 Running the T-Pot (Bellekens, 2022) 31
Figure 2.3.3.4 T-Pot Kibana Dashboard (Bellekens, 2022) 31
Figure 3.1: Agile Methodology Diagram (GeeksforGeeks, 2023) 33
Figure 3.5.1.1.1 Role in the Organisation 40
Figure 3.5.1.1.2 Type of Business 41
Figure 3.5.1.1.3 Number of Employees 41
Figure 3.5.1.1.4 Use of Wi-Fi or Connected Devices 42
Figure 3.5.1.1.5 Existence of Network Monitoring Systems 42
Figure 3.5.1.1.6 Description of Existing Security Measures 43
Figure 3.5.1.1.7 Prior Cybersecurity Incidents 44
Figure 3.5.1.1.8 Self-Rated Network Security Level 44
Figure 3.5.1.1.9 Level of Concern about Cyberattacks 45
Figure 3.5.1.1.10 Level of Concern about Cyberattacks 45
Figure 3.5.1.1.11 Presence of a Dedicated IT Department 46
Figure 3.5.1.1.12 Interest in a Low-Cost Detection Device 47
Figure 3.5.1.1.13 Importance of a Simple Dashboard 47
Figure 3.5.1.1.14 Primary Concerns about Adopting a New Security Device 48
Figure 3.5.1.1.15 Additional Comments 49
Figure 3.5.2 Use case diagram of the proposed ESP32-based IoT honeypot system 52
Figure 3.5.3 Process flowchart of the proposed honeypot detection pipeline 54

List of Tables

Table 2.2 Chronological Survey of Honeypot-Related Literature (2022–2026) 23
Table 3.4.3 Non functional requirements 38
Table 3.4.4.1 Hardware Requirement 39
Table 3.4.4.2 Software Requirements 39
Table 3.5.1.2 Interview analysis table 50

Acknowledgements

In the name of Allah, the Most Gracious and the Most Merciful. First and foremost, all praise and gratitude to Allah SWT for His blessings, strength, and guidance, which granted me the patience and determination to successfully complete this Final Year Project.

My deepest gratitude goes to my beloved parents and family for their unconditional love, endless prayers, and constant emotional and financial support throughout my academic journey.

I would like to express my highest appreciation to my project supervisor, Sir Muhamad Syafiq Bin Mohd Yasin, for his invaluable guidance, constructive feedback, and immense patience throughout the duration of this project. His expertise and encouragement were pivotal in shaping the direction of this work. I am also thankful to Madam Norfazlina Binti Johar, the FYP Coordinator, for ensuring a smooth process and providing clear milestones for our deliverables.

A special note of thanks goes to my client, Yumplingy Enterprise, for trusting me with this project, providing the necessary resources, and offering real-world insights that made this research meaningful.

I am also highly indebted to Majlis Amanah Rakyat (MARA) for their financial support and sponsorship throughout my studies, which allowed me to focus entirely on my academic pursuits. Lastly, thank you to my friends and peers for their help and motivation during this journey.

1 INTRODUCTION

1.1 Introduction

Small and medium-sized enterprises increasingly depend on internet-connected devices to run day-to-day operations such as processing orders, communicating with suppliers and handling customer payments, yet many of these businesses operate their networks without any visibility into who is attempting to access them. This chapter introduces the proposed Final Year Project, an ESP32-Based IoT Honeypot System for Intrusion Detection and Security Monitoring, which is being developed for Yumplingy Enterprise, a small dessert and food business that recently installed a Wi-Fi network to support its daily operations. The chapter begins by presenting the background of the project and the client organisation, followed by the problem statement that motivates the work, the objectives the project intends to achieve, the scope and limitations of the system, and the intended target users.

1.2 Project Background

The increase of inexpensive networking hardware has allowed companies as small as one-person operations to maintain connected environments without hiring dedicated IT teams. An example of these businesses is Yumplingy Enterprise, a local food and dessert business that manages an office physically but uses connected devices to manage orders, supplier communication and requests, customer comments, and transaction processing. This increased connectivity provides obvious operational benefits, but it also has the effect of exposing the business to many of the same types of networked threats that larger organisations have to deal with – only now without their budget, staff or skills to fend them off.

Studies on the cybersecurity state of small and medium businesses consistently show that this part of the spectrum is systematically under-protected. It is the case too with many such organisations (including those beyond just tech) where they leave their networks running without real time monitoring, resulting in them being oblivious to reconnaissance activity like port scanning or credential-guessing attempts before these culminate into a total compromise of the organisation (Chidukwani et al., 2022) This is relevant as automated reconnaissance is the first step on what tends to be a larger attack chain; an attacker who can free scan a network now knows exactly how to focus on a particular vulnerable service, device or account.

A honeypot is a type of security system which takes care of this visibility black hole by actively push out an intentional fake system onto the network. Instead of attempting to stop every possible intrusion attempt, a honeypot purposefully entices unauthorized connections and allows the attacker to interact with the decoy while recording that interaction for further analysis (Morić et al., 2025). With a properly isolated honeypot having no financial business value, just about any interconnect made along with it may be confidently treated as ill-conceived where conventional monitoring tools are plagued by ambiguity. This approach has been

shown to provide a useful method for supplying organisations with threat intelligence and forensic evidence in cases where they have no log of the attack occurring (Negi et al., 2020).

This project is about a product which applies the honeypot idea to small office limitations. The system utilizes a single ESP32 microcontroller to imitate a vulnerable Internet of Things device on the office network, rather than simulating an actual dedicated server or virtual machine, used in typical honeypot frameworks. It listens for connections to port 23 on a fake Telnet service, and a fake HTTP administration panel on port 80 — both hallmarks of automated scanning tools in the wild and altogether realistic bait for an attacker trying to probe the network. Each connection attempt to these decoy services is marked as an potential unauthorised usage, assigned to a technique and sent for structured storage and presentation. This grounds the proposed system in the context of other recently researched lightweight, IoT-class honeypot hardware solutions which utilize small embedded computing platforms at network edges to capture real-world attacker behaviours inexpensively compared to server based deployments.

Several terms are used consistently throughout the report and are defined here to reduce ambiguity in these aspects. A honeypot is a computer system or service that has no legitimate production function and exists only to log access attempts. Intrusion detection is the identification of activity on a network that suggests an attempt to compromise the security of that network (an attempted or to be actual breach). This approach is known as port scanning, it involves systematically sniffing a series of ports to find out what services are being run and can be vulnerable. A brute-force attack is the repeated automated attempt to guess a valid user name and password. Note that this background discussion is restricted to network-layer reconnaissance and credential-guessing behaviour targeting a small office network, and excludes malware analysis, endpoint protection or enterprise-grade security information and event management, which are beyond the resources and scope of the client organisation.

1.3 Problem Statement

Yumplingy Enterprise has an active office Wi-Fi network today that supports simple business activity done on computers in the office, but it was merely installed for connection — not security. There is no way to discover, record or investigate unauthorized access attempts and therefore the current security landscape of that business remains passive. The benefit of this architecture is its ease of use; staff can plug into their devices and get on with their work without having to engage with any security tooling. The flip side of that simplicity is network-level blindness in the office: No party currently looks at what goes beyond the basic security posture provided by default routing. It creates an obvious chasm between the amount of connectivity that the business has embraced and the duty needed to monitor that connectivity securely.

1.3.1 Lack of network security monitoring

Without a dedicated detection mechanism, threats such as port scanning, repeated connection attempts, and brute-force login attacks can occur on the office network entirely undetected. This is not unique to Yumplingy Enterprise; research has shown that most small and medium-sized enterprises, including technology companies, operate their networks without real-time monitoring, which leaves them vulnerable to intrusion against both internal systems and customer data (Chidukwani et al., 2022).

1.3.2 Growing cyber threats targeting office networks

Port scanning, brute-force login attempts and unauthorized devices joining a network have become common attack techniques against hearts of countless enterprises: the unfortunate fact is that they tend to be automated and indiscriminate so it hardly matters if you are an SME or a multinational entity. Statistically, small businesses are the victim of over 43% of all cyberattacks (Verizon, 2023), and in most cases can go unnoticed with no monitoring system in place for scanning activity which is usually the first phase of an attack.

1.3.3 No mechanism to capture and analyse attack behaviour

Except, if an attack were to happen today, Yumplingy Enterprise would have no means of watching, logging and analysing the behaviour of the attacker. This means you would have no forensic evidence available to identify patterns of attack in the case of a security incident, nor use the resulting data to make better security decisions. This is exactly the void that a honeypot based system aims to bridge, by providing an enticing malware free environment which can engage attack attempts with detailed logging of analysis writing usable threat intelligence while limiting the exposure of real business systems (Negi et al., 2020).

The solution of these problems is not only important for Yumplingy Enterprise. The business administers sensitive customer/financial data via its interlinked devices, and a successful breach obviously poses a threat to data security – including loss of trust in its customers, and potentially

costly mitigation efforts: both financially as well as damaging their corporate reputation beyond the scale of their actual business. Thus, the three problems above can be distilled into a general guiding research problem: given an inexpensive and simple enough solution to actually deploy and maintain without specialist support, how can a small business with no dedicated information technology staff gain real-time insight (and forensic evidence) of unauthorized access attempts on its office network?

1.4 Project Objectives

In direct response to the problems identified above, this project pursues the following three objectives. Each objective is stated using a measurable action verb so that its achievement can be verified during testing and evaluation.

1.4.1 To develop an ESP32-based honeypot device that simulates a vulnerable IoT device and monitors the office network for unauthorised access attempts.

This goal directly solves the problem of no network monitoring in Yumplingy Enterprise by creating an easily visible decoy device on his office's internal mesh. Advertising a fake Telnet service and an HTTP administration panel, the ESP32 creates an enticing target for unauthorized access attempts without having to modify or even touch the actual client's business systems. This is accomplished by creating the ESP32 firmware and represents the first line of defense as no detection or logging can take place without a decoy device being in position to lure attack traffic beforehand.

1.4.2 To detect and classify common cyber attack techniques, including port scanning, brute-force login attempts, and unauthorised connection attempts, in real time.

Automated attacks on the networks of small businesses are becoming more common, and this goal directly combats that trend by allowing a system to recognize different attack techniques rather than just generating one broad alert. With real-time detection and classification of port scanning, brute-force login attempts, or unauthorized connection attempts from individual IPs/hosts in Yumplingy Enterprise not only can it identify that an intrusion attempt occurred but also what kind of technique was used to mount the attack which is a prerequisite for responding appropriately. This is accomplished with the attack classification logic that gets run on the ESP32 and by testing simulated attacks using nmap and telnet, to validate.

1.4.3 To log detected attack evidence into a structured database and display it through a web dashboard with alerts and forensic reporting.

This goal directly solves the problem of not having any mechanism to record and analyze attacker behavior by ensuring every event detected is retained for forensic analysis forever instead of lost at the time it happens. Store information such as the source ip, timestamp of access, accessed port type of attack and payload into a structured MySQL database with all evidence presenting through Django web dashboard for alerts and basic forensic reporting; combining these functions has created Yumplingy Enterprise to evoke permanent persistent structure that can be queried later on by any staff member who does not have technical or IT backgrounds whilst still being able analyze attacker behaviours over time effectively. As part of his/her work, this objective is done by building the Django backend and creating a database + Dashboard.

1.5 Scope and Target User

This section defines the boundaries within which the proposed ESP32-Based IoT Honeypot System is developed and evaluated, outlines what the project and its final product cover, and identifies the intended users of the completed system.

1.5.1 Project Scope

The project includes a design and development of the following three integrated parts: ESP32 firmware emulating decoy Telnet and HTTP services that forwards detected events to backend; Django REST API + MySQL database receiving + storing attack evidence; Django web dashboard showing real-time statistics, attack logs & alerts. Development is run using an Agile methodology broken down into plan, design, develop, test deploy review phases allowing the system to be built and validated incrementally with required changes determined as client feedback is received.

Moreover, the project is restricted to Intentionally scoped to a low-interaction honeypot focused on the limits of single microcontroller memory and processing this system is not attempting to provide full shell-level interaction with an adversary nor does it attempt or claim functional evaluation as a complete intrusion prevention system, malware analysis sandbox (pentest information retrieval) or enterprise security information and event management platform. Now the decoy services we implemented are limited to Telnet and HTTP, hence other protocol attacks targeting them would be outside of what this implementation can detect. At the same time, its effectiveness has not been assessed in expanded corporate networks (greater than a single office) and multi-location sites outside of Yumplingy Enterprise.

1.5.2 Product Scope

The end product is a working solution that could be deployed as an actual IoT honeypot with the system combined by three deliverables acting together in one integrated pipeline. The first is ESP32 microcontroller running a fake telnet service on port 23 and an HTTP administration panel to capture login credentials from the device exposed at port 80, everything giving a convincing honeypot to any device scanning your local office network. The second is a MySQL-backed Django REST API, which handles incoming attack events forwarded by the ESP32 devoid of processing overhead, allowing event information to be entered into an organized schema where we can query source IP address timestamp accessed port type payload. The third is a Django web dashboard displaying this stored data live such as attack statistics, filterable events and alerts when newly detected activity.

Product coverage identifies and classifies three types of attack: port scanning, brute-force login attempts, or unauthorized connection attempts. It does not include malware capture, deep packet inspection or any method of active response/blocking.

1.5.3 Target User

Since the complete system is expected to be used by Yumplingy Enterprise management and staff, who are not a set of specialised networking or cybersecurity expertise. Thus, the Django dashboard is specifically tailored to display detection outcomes clearly and in a very non-technical way so that the client can confirm whether an attack actually occurred or not as well as its rough nature (without having analyzed raw logs nor packet captures).

The client side of the project is also Yumplingy Enterprise itself, and included supplying operational requirements, live office network environment used for deployment and testing as well as user acceptance-testing feedback. This is also why the system is focused around that user profile — non-technical, operating a SME (small medium enterprise), and needing actionable security information without the need for dedicated IT support to try and interpret what “threats” mean.

Other than that, this honeypot surely expects an attacker as a user. Attackers can attack the honeypot on those two services.

Admin would be able to manage, edit and delete the data in the log. It would be the Yumplingy admin or higher up to be the administrator in this product.

1.6 Conclusion

This chapter has presented the background of the proposed ESP32-Based IoT Honeypot System, established the problems that motivate the project, namely the absence of network monitoring, the growing prevalence of automated attacks against small businesses, and the lack of any mechanism to capture forensic evidence, and translated those problems into three measurable objectives. It has also defined the boundaries of the project in terms of scope, limitations, and target users. Having established why the project is needed and what it intends to achieve, the next chapter investigates the requirements of the client in more detail and surveys existing literature and honeypot solutions in order to position the design choices made for this project.

2 LITERATURE REVIEW

2.1 Introduction

This chapter investigates the body of work that informs the design of the proposed ESP32-Based IoT Honeypot System. Rather than summarising individual sources in isolation, the chapter critically examines the relationships between recent studies on honeypot-based intrusion detection, several existing honeypot frameworks, and the specific operating constraints of Yumplingy Enterprise, in order to identify the gap that this project is intended to fill. The chapter begins with an investigation of the client's requirements and the technical approach taken to researching the project, structured using the What, Where, Who, Why, and How method, followed by a chronological survey of recent academic literature on honeypot-based detection. It then reviews three existing honeypot frameworks, namely Honeyd, Cowrie, and T-Pot, in detail before comparing them directly against one another and against the system proposed in this report. The chapter concludes with a discussion that draws together the literature gap, the related-works comparison, and the design decisions carried forward into this project.

2.2 Investigation

This section presents the investigation carried out to support the project, beginning with a structured What, Where, Who, Why, and How analysis of the client's requirements, followed by a conceptual investigation of honeypot-based intrusion detection itself, covering its definition and mechanism, its types, the attack techniques it is designed to detect, and real-world case studies of comparable low-cost deployments, and closing with a chronological survey of the academic literature that grounds the design decisions made in this report.

What. This paper investigates honeypot and intrusion-detection methods suitable for a small office network without any dedicated IT team, as well as the technologies needed to implement such an approach on inexpensive off-the-shelf microcontroller hardware. This includes studying how honeypot frameworks have simulated vulnerable services as well as examining what the common network attack types a system such as port scanning, brute-force login attempts and unauthorised connections are typically executed and recognised, concluding with demonstrating how those components can be assembled together — integrating an ESP32 platform (a lightweight microcontroller chip), Django web framework and MySQL database to form a streamlined detection pipeline for forensic logging.

Where. User needs were investigated at Yumplingy Enterprise's physical office, where the company's Wi-Fi network actually runs and where the completed system will be deployed live to enable monitoring and user acceptance testing. The supporting technical investigation was conducted separately via official documentation (Espressif ESP32, n.d.), project-based documentation for the Django and MySQL application

stack as well OWASP Internet of Things guidelines (OWASP Foundation, n.d.) while considering relevant published academic literature on honeypot-based detection.

Who. Directly involved in the investigation are two groups of stakeholders. The first of these is Yumplingy Enterprise themselves, as captured by their management and IT-facing staff both being consulted to better understand the existing situation in regard to office network dependency along with what devices/tasks depend on it working at all; expectations regarding a provisioned future monitoring solution. The second group is composed of the researchers and maintainers associated with the literature, as well as tools covered in this chapter; more precisely those behind each of the four academic studies evaluated in Section 2.2, plus designers for Honeyd (Provoset al., 2004), Cowrie or T-Pot (Telekom Security et al., 2023) whose actual findings are thoroughly identified through documentation built up their weakness/espectations directly guide to inform architecture intended by running project on that base line

Why. This investigation is important because small businesses such as Yumplingy Enterprise face a disproportionate share of automated attacks while having the least capacity to detect them (Verizon, 2023; Chidukwani et al., 2022), and because, as the literature survey below demonstrates, the honeypot research and tools that already exist are, almost without exception, evaluated and built for environments with access to dedicated servers, virtual machines, or full Linux hosts. Without this investigation, there is a risk of either under-engineering a solution that fails to detect realistic attack techniques, or over-engineering a solution that the client cannot afford, install, or maintain.

How. This was done as a technical investigation of existing honeypot frameworks that had successfully implemented features such as service emulation, attack logging and visualisation to prepare the design patterns against deliverable hardware/software from the client. The target implementation targets an ESP32 development board, programmed in the Arduino IDE with C++, to provide fake Telnet and HTTP services; a Django REST API server as receiver of detection events; and a MySQL database which stores detections to allow structured, queryable storage. Requirements gathering with the client involved a structured interview addressing current office network and existing security measures (supported by distribution of a short questionnaire among office staff to assess their day-to-day use in connected devices), as well attack simulation during testing, which was conducted via nmap and Telnet from Kali Linux machine for realistic reconnaissance detection accuracy validation against brute-force behaviour.

2.2.1 Definition and Mechanism of Honeypot-Based Intrusion Detection

Before existing tools and systems are reviewed in Section 2.3, this sub-section first establishes the underlying concept the proposed system is built on: what a honeypot is, and the mechanism through which it actually detects an intrusion.

A honeypot is a decoy computer system or network service that is deliberately exposed to attackers and has no legitimate production function, so that any interaction with it can be treated as evidence of unauthorised activity (Morić et al., 2025). Honeypot technologies provide valuable insight into adversary behaviour while maintaining a low rate of false detections, since genuine users have no reason to ever connect to a decoy, and they generate forensic evidence and usable threat intelligence for organisations that would otherwise have no record that an attack had taken place at all (Negi et al., 2020; Javadpour et al., 2024).

Literature consistently describes honeypot-based detection as having three clearly distinguishable stages, instead of a single interaction. The first is deception: A honey-pot service that looks like a legitimate vulnerable system, attracting scanning and credential-guessing traffic which would otherwise be going to real infrastructure. Second is interaction capture: any link, login attempt or command sent to the decoy will be intercepted and logged. Third, analysis and classification: events that have been captured are analysed, harvested for useful Threat Intelligence (and classified) enabling the feedback loop closing detection to informed response (Javadpour et al., 2024).

We structure the ESP32 honeypot to follow the same three stages: deception by fake Telnet and HTTP services, interaction capture by Django REST API, analysis / classification with MySQL-backed dashboard.

2.2.2 Types of Honeypots by Interaction Level

Honeypots are not a single uniform technology; the literature consistently classifies them by how much genuine interaction they allow an attacker to have, and this classification has a direct bearing on what hardware a honeypot can realistically run on.

- Low-interaction honeypots emulate only the surface-level behaviour of a service, such as a login banner or a handful of expected responses, which makes them lightweight and easy to deploy at scale, and particularly effective at capturing early-stage activity such as automated scans and brute-force attempts (Morić et al., 2025).
- Medium- and high-interaction honeypots, by contrast, emulate a much fuller environment, such as an interactive shell or a complete operating system, which captures richer attacker behaviour but at a correspondingly higher resource and maintenance cost (Morić et al., 2025).

- This produces a direct trade-off between deployment cost and forensic depth: low-interaction tools scale well but produce weaker data quality, while medium-interaction tools capture richer detail at higher resource cost (Morić et al., 2025).

This justifies the proposed system's choice of a low-interaction design: a single ESP32 microcontroller cannot realistically host a medium- or high-interaction honeypot, but the literature confirms that low-interaction emulation is already sufficient to capture the specific attack types, port scanning and brute-force login attempts, that this project is designed to detect.

2.2.3 Common Attack Techniques Targeting Small Office and IoT Networks

Alongside the honeypot technology itself, the investigation also examined the specific attack techniques the proposed system is intended to detect, since a honeypot is only useful if it is designed around realistic attacker behaviour.

Port scanning**.** is the technique of systematically probing a range of network ports to discover which services are running and potentially vulnerable, and is typically the first stage of a wider attack chain (Chidukwani et al., 2022).

Brute-force login attempts involve repeated, automated attempts to guess a valid username and password combination; analysis of honeypot data has found that such attempts overwhelmingly target weak, predictable credentials rather than sophisticated exploits (Zielinski & Kholidy, 2023).

Prevalence across independently deployed honeypots, brute-force attempts and port/service scanning are consistently found to be the most prevalent categories of adversary behaviour, together accounting for almost the entirety of observed cyber bot activity in large-scale measurement, and small businesses specifically are reported to be the target of more than 43% of all cyberattacks (Wang et al., 2026; Verizon, 2023).

This justifies treating port scanning, brute-force login attempts, and unauthorized connection attempts as the three primary attack categories the proposed system is built to detect and classify, directly matching Objective 2 set out in Chapter 1.

2.2.4 Case Studies of Low-Cost and Embedded Honeypot Deployments

Because this project specifically targets microcontroller-class hardware rather than a server, particular attention was paid to case studies that deploy honeypots on similarly constrained, low-cost devices, to confirm that the concepts established above translate into a practical, working system.

- A honeypot built by integrating the Cowrie software with a low-cost Raspberry Pi 3 Model B, configured to simulate an IoT device, successfully captured the most exploited credentials, the most frequently used shell commands, and the geographic origin of real attacks,

demonstrating that genuine forensic value does not require expensive infrastructure (Valentini et al., 2025).

- A T-Pot deployment evaluated specifically for network intrusion detection performance confirmed that structured logging combined with dashboard-based analysis substantially improves an operator's ability to interpret captured attack data compared with raw log inspection (Mohd Faris Mohd Fuzi et al., 2024).

This supports both the choice of low-cost embedded hardware as a credible honeypot platform and the decision to store captured fields, credentials and payload, source IP address, and geographic origin where available, in a structured MySQL schema rather than plain text logs.

2.2.5 Challenges and Limitations of Honeypot-Based Detection

The literature is equally clear that honeypot-based detection carries real limitations that must be designed around rather than ignored.

- Once a sophisticated attacker identifies the presence of a deception mechanism, they may alter their tactics or disengage entirely, meaning honeypots must be carefully maintained to remain convincing over time (Javadpour et al., 2024).
- Low-interaction honeypots, the category best suited to constrained hardware, consistently trade away data depth and quality in exchange for scalability and low resource cost (Morić et al., 2025).
- Resource-constrained embedded platforms cannot realistically emulate a full operating system or filesystem, which limits the range of attacker behaviour that can be captured compared with a high-interaction tool running in full proxy mode.

Countering these limitations, the proposed ESP32 Honeypot System deliberately simplifies its design to focus on two well-known automated attacks consisting of Telnet and HTTP protocols rather than attempting broad but less manageable deceptions and by keeping any connection made to the decoy an anomaly so that classification remains easy enough such as it can run reliably even in constrained hardware environments.

The literature survey is supplementary to the concept investigation above and was used primarily for discovering non-existent theories/methods that have been documented regarding honeypot based intrusion detection on a chronological basis. A summary of four illustrative studies covering the time period 2022 to 26, that were considered relevant as they directly address honeypot effectiveness, analysis of attacker behaviour or requirements for small organisations cybersecurity needs are summarised below in Table 2.1

***Table 2.2** Chronological Survey of Honeypot-Related Literature (2022–2026)*

Year	Title / Author(s)	Aim	Findings
2026	Mapping Cyber Bot Behaviors: Understanding Payload Patterns in Honeypot Traffic (Wang et al., 2026)	To map cyber bot behaviour through an 11-month, 50-honeypot measurement study, automatically extracting and labelling recurring payload patterns across 21.5 million captured packets.	Found that service/port scanning (78.1%) and brute-force login attempts (20.0%) together accounted for the overwhelming majority of observed cyber bot behaviour, far ahead of web crawling, vulnerability scanning, or exploitation.
2025	Advancing Cybersecurity with Honeypots and Deception Strategies (Morić, Dakić & Regvart, 2025)	To develop a systematic framework for comparing honeypot solutions, including Honeyd and Cowrie, across detection scope, scalability, reliability and data quality.	Found a direct trade-off between deployment cost and forensic depth: low-interaction tools such as Honeyd scale well but produce weaker data quality, while medium-interaction tools such as Cowrie capture richer attacker detail at higher resource cost.
2023	An Analysis of Honeypots and Their Impact as a Cyber Deception Tactic (Zielinski & Kholidy, 2023)	To deploy a Cowrie-based honeypot system, test it using Nmap port scanning and Hydra brute-force attacks, and analyse three weeks of real attacker data.	Found that the Cowrie SSH honeypot was the single most attacked service in the deployment, that attackers overwhelmingly attempted weak or default credentials such as root/admin, and that attacking traffic was geographically concentrated in a small number of countries.
2022	A Survey on the Cyber Security of Small-to-Medium Businesses: Challenges, Research Focus and Recommendations (Chidukwani, Zander & Koutsakis, 2022)	To survey the cybersecurity challenges faced by small and medium businesses and identify gaps between academic research focus and SME needs.	Found that most SMEs operate without real-time network monitoring and that security research disproportionately targets large enterprises, leaving a gap for low-cost monitoring aimed at small businesses.

Taken together these four studies show coherence, not a series of disparate findings. Wang et al. (2026) and Morić et al. establish that brute-force login attempts and reconnaissance-style port or service scanning are by far the most prevalent (and detectable) attack categories across many independent honeypot deployments, fundamentally justifying treating these two techniques (together with unauthorised connection attempts) as the key detection fields of interest for our system. Zielinski and Kholidy (2023) also demonstrates that this behaviour of an attacker is realistically replicable using light interaction strategies, e.g., Telnet/SSH emulation based on unprocessed cowrie-type shell rather than a full high-interaction shell which shows the rationale behind simple decoys in the project consisting just from simpletelnet / HTTP while not going with heavier environment to reproduce them. Concurrently, wider comparative work by Morić et al. The documentation also includes the fact that Chidukwani et al. focused on a survey geared towards small business owners (2025). (2022) reveal an open question rather than a bankruptcy to the solution space: published honeypot reviews continually compromise detection depth for deployment cost assuming server-elegance infrastructure, and protection research extra widely maintains its widespread neglect of small agencies devoid of information generation personnel. None of the four tested a honeypot on an affordable microcontroller-class hardware platform for < RM 25, nor coupled attack detection with a lightweight forensic dashboard that facilitates non-technical users. It is speculative to state that the proposed ESP32-based honeypot can indeed be deployed in resource-constrained small-and-medium enterprises (SMEs) but, this specific combination of low-cost detection hardware combined with simplified forensic reporting for these institutions is however what we are aiming at and will explore further through our related-works comparison in sections 2.3.

2.3 Related Works

This section reviews three existing honeypot frameworks identified as the most relevant prior art for this project: Honeyd, a long-established low-interaction framework; Cowrie, a widely used medium-interaction SSH and Telnet honeypot; and T-Pot, a comprehensive multi-honeypot platform. Each framework is examined in terms of its definition, operating mechanism, key features, security mechanism, and advantages and disadvantages, illustrated with a representative figure constructed for this report based on each framework's documented usage.

2.3.1 Honeyd

Honeyd is an open-source virtual honeypot framework that simulates the network-level presence of multiple virtual computer systems from a single physical machine, allowing a defender to populate otherwise unused network addresses with convincing decoys (Provos, 2004).

Honeyd runs a configurable personality engine that mimics the TCP/IP stack behaviour of different operating systems for each virtual host it creates. When a scanning tool such as Nmap probes the network, it perceives each configured virtual address as a distinct machine running its own operating system and open services, even though only one physical host is actually responsible for generating every response.

Configurable virtual network topologies; arbitrary TCP, UDP, and ICMP service emulation through external scripts bound to specific ports; operating-system personality spoofing to defeat fingerprinting tools; and the ability to populate large unused address ranges with decoys from a single physical or virtual host.

Honeyd's mechanism is deception at the network and transport layer: by convincing scanning tools that decoys are genuine, differentiated machines, it diverts and exposes reconnaissance activity that would otherwise go unnoticed, and has previously been used to detect and slow the spread of network worms (Provos, 2004).

Honeyd is highly scalable, since a single host can convincingly represent a large number of virtual systems; it is flexible enough to emulate almost any service or network topology; and it is a long-established, well-documented framework with a substantial body of literature behind it.

Honeyd requires a dedicated server or virtual machine together with non-trivial manual template and service configuration, which is impractical for a small office with no server infrastructure or in-house networking expertise. Its data quality is also limited to relatively basic connection-level information, a limitation independently confirmed by Morić et al. (2025), who found that Honeyd's scalability comes at the cost of weaker data quality compared with medium-interaction tools.

```
$ honeyd -f honeyd.conf -i eth0
create template
set template personality "Windows XP"
add template tcp port 23 "sh scripts/telnet.sh"
bind 10.0.0.5 template

honeyd[2381]: Listening on 10.0.0.5
> connection from 192.168.1.42: TCP 23 SYN-RECV
```

Figure 2.3.1.1 *Illustrative example of Honeyd configuration commands and a connection log entry, constructed for this report based on the documented usage described in Provos (2004).*

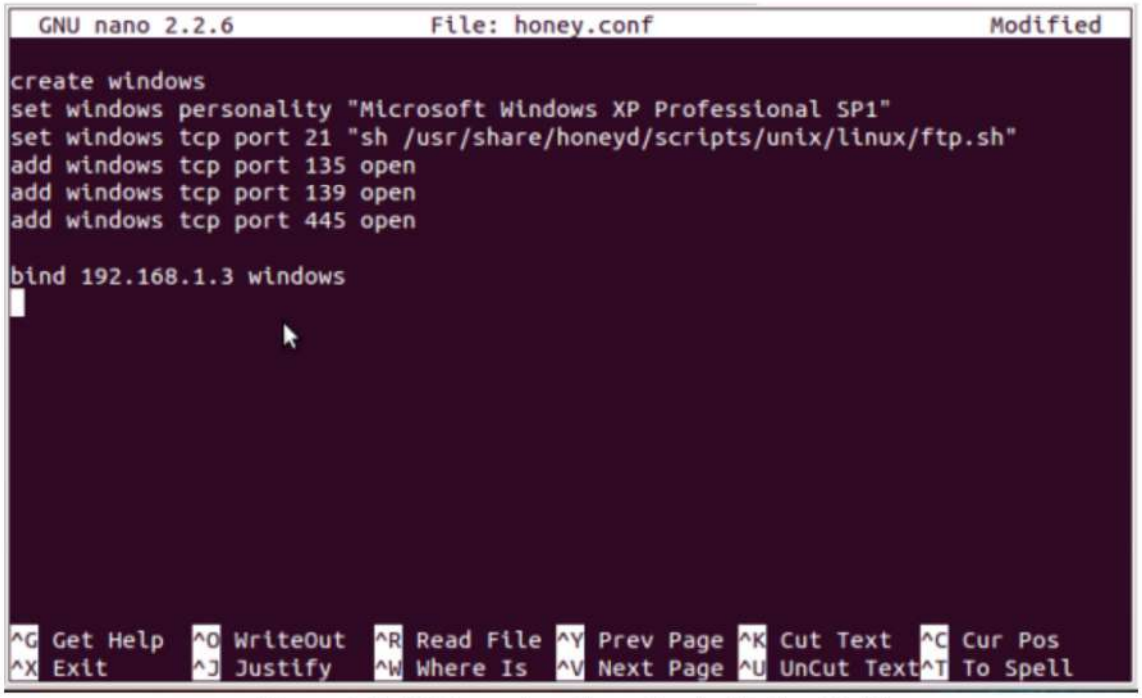


Figure 2.3.1.2 *Honeyd configuration by (Gajjar, 2013)*



Figure 2.3.1.3 *Running the honeyd service (Gajjar, 2013)*

2.3.2 Cowrie SSH/Telnet Honeypot

Cowrie is a medium-to-high interaction honeypot purpose-built to log brute-force attacks and the shell-level interaction performed by an attacker once a session is established.

In its medium-interaction mode, Cowrie emulates a Unix-like shell environment in Python, intercepting and responding to attacker commands without executing them on a real system; in its high-interaction proxy mode, it instead forwards the attacker's session to another real system for direct observation.

Emulated SSH and Telnet services; a simulated filesystem and command shell that responds to common Linux commands; capture of usernames, passwords, and full command sequences; and structured session logging suitable for later analysis.

Cowrie's mechanism centres on credential and session capture: every login attempt, command, and file operation an attacker performs inside the emulated shell is recorded for later analysis, which directly supports threat intelligence on attacker tactics and intent.

Cowrie captures unusually rich, realistic detail about attacker interaction, since a convincing emulated shell encourages attackers to reveal more of their intent than a simple connection banner would. A deployment that built a Cowrie-based honeypot system and analysed three weeks of real attack traffic found that Cowrie's SSH service was the single most attacked component of the entire system, with attackers overwhelmingly attempting weak or default credentials and originating from a geographically concentrated set of countries, demonstrating the practical value of the data Cowrie can capture (Zielinski & Kholidy, 2023).

Cowrie depends on a full Linux host with a Python runtime and a reasonably complete filesystem emulation, making it considerably more resource-intensive than the project's ESP32 hardware budget allows, and it provides no built-in dashboard for non-technical users to interpret its logs.

```
$ cowrie start
Cowrie SSH/Telnet honeypot listening on :2223

login attempt: admin / admin123
login attempt: root / 123456  -> accepted
$ cat /etc/passwd
$ wget hxxp://185.23.x.x/payload.sh
session closed -> log/cowrie.json
```

Figure 2.3.2.1 *Illustrative example of a Cowrie login attempt and emulated shell session, constructed for this report based on the deployment behaviour described in Zielinski and Kholidy (2023).*

```
# Package generated configuration file
# See the sshd_config(5) manpage for details

# What ports, IPs and protocols we listen for
Port 3393
# Use these options to restrict which interfaces/protocols sshd will bind to
#ListenAddress ::
#ListenAddress 0.0.0.0
Protocol 2
# HostKeys for protocol version 2
HostKey /etc/ssh/ssh_host_rsa_key
HostKey /etc/ssh/ssh_host_dsa_key
HostKey /etc/ssh/ssh_host_ecdsa_key
HostKey /etc/ssh/ssh_host_ed25519_key
#Privilege Separation is turned on for security
UsePrivilegeSeparation yes

# Lifetime and size of ephemeral version 1 server key
KeyRegenerationInterval 3600
ServerKeyBits 1024

# Logging
SyslogFacility AUTH
LogLevel INFO
```

Figure 2.3.2.2 *Configuration of cowrie (French, 2018)*

```
2016-09-24T11:59:48-0700 [SSHService 'ssh-userauth' on HoneyPotSSHTransport,1,[redacted]] 'root' trying auth 'none'
2016-09-24T11:59:48-0700 [SSHService 'ssh-userauth' on HoneyPotSSHTransport,1,[redacted]] 'root' trying auth 'keyboard-interactive'
2016-09-24T12:00:00-0700 [SSHService 'ssh-userauth' on HoneyPotSSHTransport,1,[redacted]] login attempt [root/strongpassword] succeeded
2016-09-24T12:00:01-0700 [SSHService 'ssh-userauth' on HoneyPotSSHTransport,1,[redacted]] 'root' authenticated with 'keyboard-interactive'
2016-09-24T12:00:01-0700 [SSHService 'ssh-userauth' on HoneyPotSSHTransport,1,[redacted]] starting service 'ssh-connection'
2016-09-24T12:00:01-0700 [SSHService 'ssh-connection' on HoneyPotSSHTransport,1,[redacted]] got channel 'session' request
2016-09-24T12:00:01-0700 [SSHChannel session (0) on SSHService 'ssh-connection' on HoneyPotSSHTransport,1,[redacted]] channel open
2016-09-24T12:00:01-0700 [SSHChannel session (0) on SSHService 'ssh-connection' on HoneyPotSSHTransport,1,[redacted]] pty request: 'xterm' (45, 77, 0, 0)
2016-09-24T12:00:01-0700 [SSHChannel session (0) on SSHService 'ssh-connection' on HoneyPotSSHTransport,1,[redacted]] Terminal Size: 45 77
2016-09-24T12:00:01-0700 [SSHChannel session (0) on SSHService 'ssh-connection' on HoneyPotSSHTransport,1,[redacted]] getting shell
2016-09-24T12:00:01-0700 [SSHChannel session (0) on SSHService 'ssh-connection' on HoneyPotSSHTransport,1,[redacted]] Opening TTY Log: log/tty/20160924-1
20001-37f30bd9-0i.log
2016-09-24T12:00:04-0700 [SSHChannel session (0) on SSHService 'ssh-connection' on HoneyPotSSHTransport,1,[redacted]] CMD: pwd
2016-09-24T12:00:04-0700 [SSHChannel session (0) on SSHService 'ssh-connection' on HoneyPotSSHTransport,1,[redacted]] Command found: pwd
2016-09-24T12:00:09-0700 [SSHChannel session (0) on SSHService 'ssh-connection' on HoneyPotSSHTransport,1,[redacted]] CMD: cd /home/
2016-09-24T12:00:09-0700 [SSHChannel session (0) on SSHService 'ssh-connection' on HoneyPotSSHTransport,1,[redacted]] Command found: cd /home
2016-09-24T12:00:11-0700 [SSHChannel session (0) on SSHService 'ssh-connection' on HoneyPotSSHTransport,1,[redacted]] CMD: ls -lt
2016-09-24T12:00:11-0700 [SSHChannel session (0) on SSHService 'ssh-connection' on HoneyPotSSHTransport,1,[redacted]] Command found: ls -lt
2016-09-24T12:02:18-0700 [SSHChannel session (0) on SSHService 'ssh-connection' on HoneyPotSSHTransport,1,[redacted]] CMD: exit
2016-09-24T12:02:18-0700 [SSHChannel session (0) on SSHService 'ssh-connection' on HoneyPotSSHTransport,1,[redacted]] Command found: exit
2016-09-24T12:02:18-0700 [SSHChannel session (0) on SSHService 'ssh-connection' on HoneyPotSSHTransport,1,[redacted]] exitCode: 0
2016-09-24T12:02:18-0700 [SSHChannel session (0) on SSHService 'ssh-connection' on HoneyPotSSHTransport,1,[redacted]] sending request 'exit-status'
```

Figure 2.3.2.3 *Cowrie Log (French, 2018)*

2.3.3 T-Pot (All-in-One Honeypot Platform)

T-Pot is a complete, multi-honeypot platform that integrates more than twenty individual honeypot daemons, a network intrusion detection system, and an Elastic Stack-based backend for the visualisation of attack data (Telekom Security, 2024).

T-Pot runs each integrated honeypot service inside an isolated Docker container on the same network interface, so that many different decoy services can listen for different attack types simultaneously; captured events are forwarded into an Elastic Stack pipeline that indexes, correlates, and visualises them in near real time.

More than twenty integrated honeypot daemons covering a wide range of protocols; a network intrusion detection system layered on top of the honeypots; live Kibana dashboards; and an animated, geography-based attack map.

T-Pot's mechanism is breadth-based deception: by running many different honeypot services at once, it can attract and classify a very wide range of attack types directed at many different protocols simultaneously, rather than specialising in any single service.

T-Pot provides comprehensive threat visibility across many protocols at once, supported by mature, ready-made dashboards that make large volumes of attack data easy to interpret without additional development work.

This comprehensiveness comes at a significant resource cost: the platform's own documentation recommends at least eight to sixteen gigabytes of memory and over one hundred gigabytes of disk space, together with the technical skill needed to operate a Docker-based, multi-service Linux deployment (Telekom Security, 2024), which places it well beyond the resources and skills available to a small office such as Yumplingy Enterprise with no dedicated information technology team.



Figure 2.3.3.1 *Illustrative example of a T-Pot-style dashboard summary view, constructed for this report based on the dashboard capabilities described in Telekom Security (2024).*

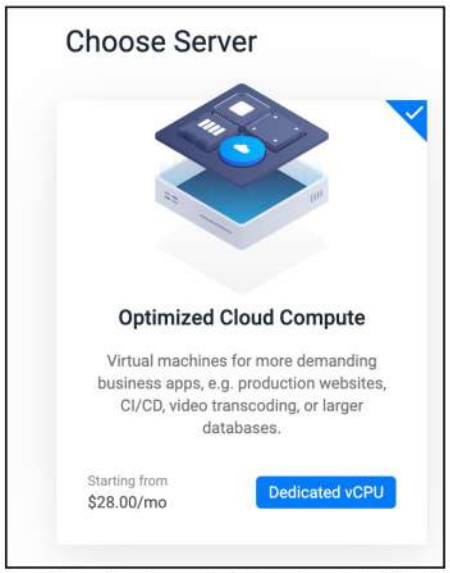


Figure 2.3.3.2 *Need to have VPS to host it (Bellekens, 2022)*

```
################################################################

### Checking for active services.

Active Internet connections (only servers)
Proto Recv-Q Send-Q Local Address           Foreign Address         State       User       Inode      PID/Program name
tcp        0      0 0.0.0.0:22              0.0.0.0:*               LISTEN      0          19884      1132/sshd: /usr/sbi
tcp        0      0 127.0.0.1:25            0.0.0.0:*               LISTEN      0          13520      1101/exim4
tcp6       0      0 :::22                   :::*                    LISTEN      0          19888      1132/sshd: /usr/sbi
tcp6       0      0 ::1:25                  :::*                    LISTEN      0          13521      1101/exim4
udp        0      0 0.0.0.0:68              0.0.0.0:*                           0          12462      668/dhclient
udp        0      0 108.61.217.135:123      0.0.0.0:*                           0          18287      3969/ntpd
udp        0      0 127.0.0.1:123           0.0.0.0:*                           0          18285      3969/ntpd
udp        0      0 0.0.0.0:123             0.0.0.0:*                           0          18281      3969/ntpd
udp6       0      0 fe80::5400:4ff:fe04:123 :::*                                0          18293      3969/ntpd
udp6       0      0 2001:19f0:6001:54dc:123 :::*                                0          18291      3969/ntpd
udp6       0      0 ::1:123                 :::*                                0          18289      3969/ntpd
udp6       0      0 :::123                  :::*                                0          18278      3969/ntpd

### Please review your running services.
### We will take care of SSH (22), but other services i.e. FTP (21), TELNET (23), SMTP (25), HTTP (80), HTTPS (443), etc.
### might collide with T-Pot's honeypots and prevent T-Pot from starting successfully.

Continue [y/n]?
```

Figure 2.3.3.3 *Running the T-Pot (Bellekens, 2022)*

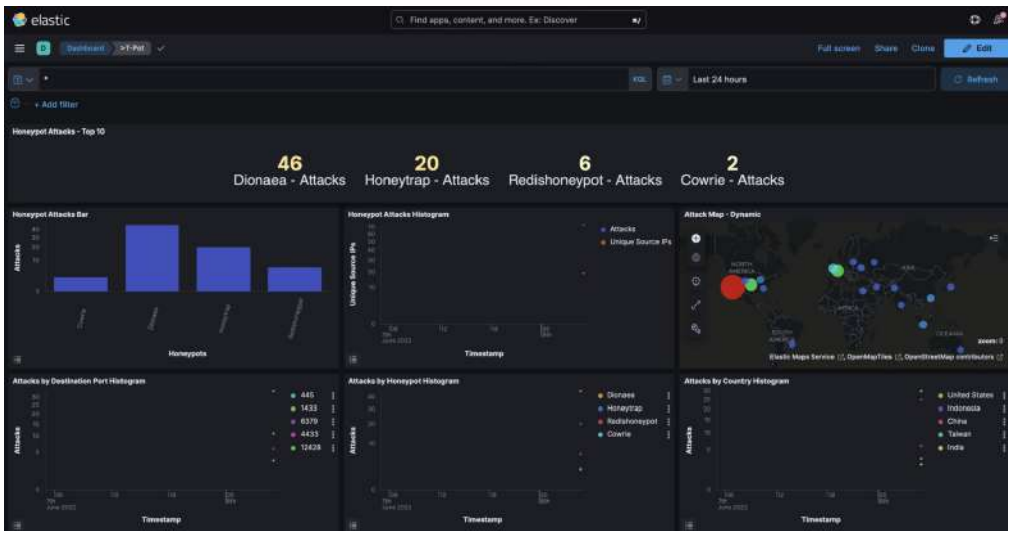


Figure 2.3.3.4 *T-Pot Kibana Dashboard (Bellekens, 2022)*

3 METHODOLOGY

3.1 Introduction

This chapter presents the methodology adopted for the development of the proposed ESP32-Based IoT Honeypot System. Building on the problems identified in Chapter 1 and the research gap established through the literature survey and related-works comparison in Chapter 2, this chapter explains how the project will be carried out in practice. The chapter begins by justifying the choice of Agile methodology and describing its phases, followed by a presentation of the hardware, software, functional, and non-functional requirements of the proposed system. The chapter then presents the analysis carried out to support the system design, including the analysis of the questionnaire and client interview, the use case model, the process flowchart, and the Business Process Modelling Notation (BPMN) diagram of the system. The chapter closes with a conclusion that links the methodology to the system design and implementation work to be carried out in the remainder of the project.

3.2 Agile Methodology

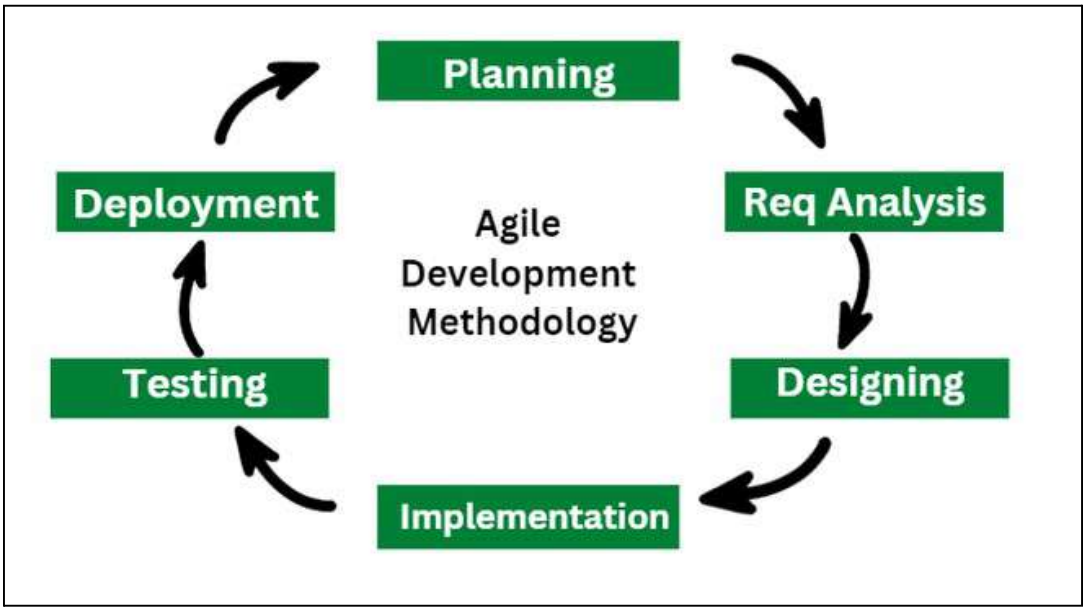


Figure 3.1: *Agile Methodology Diagram (GeeksforGeeks, 2023)*

For this project scenario, agile methodology is chosen because the system will be developed and proven using small iterations with minimum possible outputs rather than as a fixed scope deliverable in one single go. This is especially appropriate for the honeypot system we propose here, as many parts of the project such as specific thresholds (the point a conditional attack becomes an actual invasion), forensic database structure and dashboard layout may develop from repeated testing or consultations with our client Yumplingy Enterprise.

Agile allows for feedback gathered from the questionnaire and client interview in this chapter to affect design decisions on subsequent iterations, whereas a Waterfall approach would require all requirements be finalized before development starts. This is significant since the client lacks a dedicated information technology background, meaning that usability criteria (see Section 3.4.1 for dashboard simplicity) may not be clear until an early version of the Dashboard is then demonstrated to them by experienced developers/directors—which can show how their assumptions stand with respect to success under first exposure and approach-guided error-checking feedback in future interactions as well via limited-head testing ("finally! Live Test/Alpha Feedback!") Agile development lowers the technical risk from limited microcontroller hardware, as each part of your system (the decoy services, detection logic and database integration) can be developed independently.

3.3 Phases in Agile Methodology

The development of the proposed system is structured into four Agile phases, beginning with planning and progressing through design, development, and testing. Each phase produces a concrete deliverable that feeds into the next, ensuring that the system is built incrementally and validated at every stage.

3.3.1 Phase 1: Plan

The goal of the planning phase is to study what will serve both an end-user or client and a larger target audience. These cover carrying out the structured interview of Yumplingy Enterprise to determine as-is office network and its security practices, handing-out questionnaire around a larger concentration of small/mid-size enterprise staff and ownership in order to verify that this is indeed an industry-wide issue; examining literature/honeypot frameworks discussed within Chapter 2 for guiding technology choices conducted throughout project. In this phase, a well-defined set of hardware, software functional and non-functional requirements are produced as shown in Section 3.4.

3.3.2 Phase 2: Design

The Design phase makes the requirements gathered in Phase 1 tangible as a real system design. This encompasses overall system architecture between ESP32 honeypot, Django backend and MySQL database; the designed schema for forensic attack records storage, modelling of system behaviour using use case diagram, process flowchart and Business Process Modelling Notation presented in Section 3.5 respectively This approach clearly defines the relationship between all actors involved in the system, Attacker, Staff and Admin as they relate to detection (and logging) before any development begins.

3.3.3 Phase 3: Develop

Development The system is actually built in this phase based on the design generated during Phase 2. This involves programming the ESP32 firmware to drive Telnet and HTTP services as fake ones, implementing attack classification onboard; building Django REST API for communicating data; using MySQL database for structured forensic storage measures and creating Django dashboard showing live attacks logs together with statistics or alerts. Since this is an Agile process, anticipate developing and perfecting each of these elements iteratively with foundational components like the ESP32 firmware and database integration finished before more topical modules such as dashboard development or forensic reporting feature.

3.3.4 Phase 4: Test

The testing phase validates that the developed system meets the functional and non-functional requirements identified in Section 3.4. This includes unit testing of individual components, integration

testing of the complete data pipeline from the ESP32 to the database, and system testing using simulated attacks performed with tools such as nmap and Telnet from a Kali Linux machine, to confirm that port scanning, brute-force login attempts, and unauthorised connection attempts are correctly detected and classified. Feedback gathered during testing, including from a future user acceptance testing session with Yumplingy Enterprise, is intended to be fed back into subsequent design and development iterations, consistent with the iterative nature of Agile methodology.

3.4 Requirement

This section presents the requirements identified for the proposed system, beginning with the data gathering techniques used to identify these requirements, followed by the functional, non-functional, and system requirements derived from them

3.4.1 Data Gathering Techniques

Two data gathering techniques were used to identify the requirements of this project: a questionnaire distributed to a general sample of small and medium enterprise respondents, and a structured interview conducted directly with the project client, Yumplingy Enterprise.

3.4.1.1 Questionnaire

A Google Form questionnaire was distributed to 50 respondents (n = 50) consisting of staff and owners working across a range of business fields, including cafes and restaurants, tuition centres, online resellers, food and dessert businesses, and information technology services. The purpose of the questionnaire was to validate the broader problem identified in Chapter 1 by assessing the current state of network security monitoring, the level of concern regarding cyberattacks, and the level of interest in a low-cost detection solution among small businesses in general, rather than Yumplingy Enterprise alone.

3.4.1.2 Interview

Interview. A structured interview was conducted with Yumplingy Enterprise management via Google Meet. The purpose of the interview was to understand the client's specific network setup, current security measures, and expectations for the proposed system in greater depth than a general questionnaire could provide, given that Yumplingy Enterprise is the specific organisation the system is being designed to serve.

3.4.2 Functional Requirements

- The system shall simulate a vulnerable IoT device using a fake Telnet service on port 23 and a fake HTTP administration panel on port 80.
- The system shall detect and classify port scanning, brute-force login attempts, and unauthorized connection attempts in real time.
- The system shall record each detected attack event with the source IP address, timestamp, accessed port, attack type, and payload.
- The system shall store all detected attack records in a structured MySQL database.
- The system shall display live attack logs, summary statistics, and alerts through a Django web dashboard.
- The system shall generate a downloadable forensic report summarising detected attack events.
- The system shall provide an administrative panel through which stored records can be managed and reviewed.

3.4.3 Non-functional requirements

Table 3.4.3 *Non functional requirements*

Usability	dashboard must be simple enough for non-technical staff to understand without training
Performance	attack detection and dashboard update should occur in near real time (within seconds)
Reliability	ESP32 must auto-reconnect to Wi-Fi if connection drops, maintaining continuous monitoring
Affordability	total hardware cost must remain low (under RM 25) to suit SME budget constraints
Security	dashboard access secured via HTTPS; admin panel protected with authentication
Scalability (limited)	designed for a single office network; not evaluated for larger or multi-site deployments

3.4.4 System Requirements

3.4.4.1 Hardware Requirements

Table 3.4.4.1 *Hardware Requirement*

ESP32	main honeypot device
USB data cable	for flashing firmware and power
Laptop/PC	to run Django backend, MySQL database, and Arduino IDE
Wi-Fi router	connects ESP32 and backend on the same network
VPS (Virtual Private Server)	for live deployment and remote dashboard access

3.4.4.2 Software Requirements

Table 3.4.4.2 *Software Requirements*

Arduino IDE	ESP32 firmware development (C++)
Python 3	backend development
Django	web framework for REST API and dashboard
MySQL / MariaDB	relational database for attack log storage
Docker	containerisation for deployment
Nginx	reverse proxy and HTTPS handling on VPS
Kali Linux / nmap / Telnet	used for simulating attacks during testing

3.5 Analysis

This section presents the analysis carried out based on the data gathering techniques described in Section 3.4.1, followed by the modelling of the proposed system through a use case model, a process flowchart, and a Business Process Modelling Notation (BPMN) diagram.

3.5.1 Data Gathering Analysis

3.5.1.1 Questionnaire Analysis

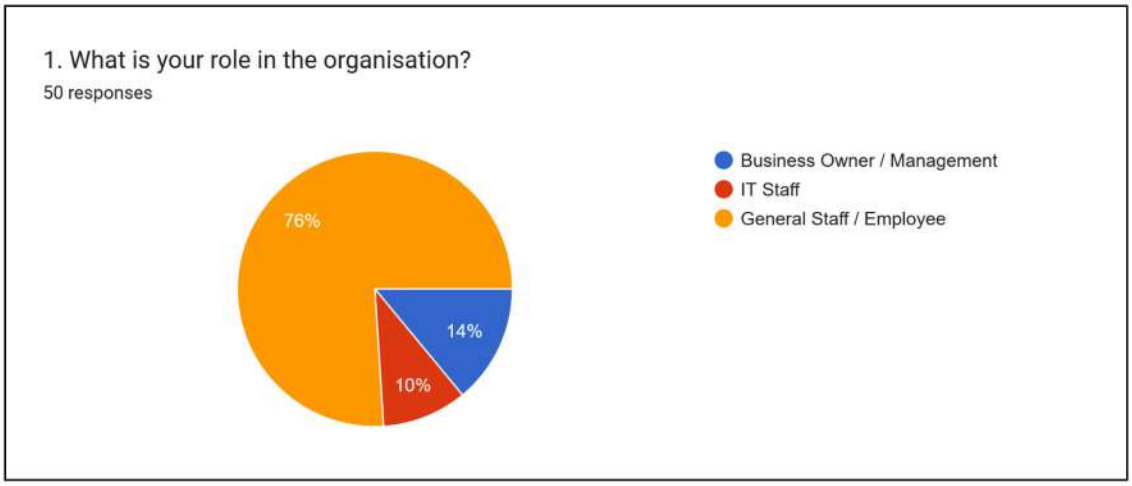


Figure 3.5.1.1.1 *Role in the Organisation*

Out of 50 respondents, 76% (38) identified as General Staff/Employees, 14% (7) as Business Owners/Management, and 10% (5) as IT Staff. This distribution is consistent with the typical organisational structure of micro and small enterprises, where front-line staff substantially outnumber management and technical personnel. The skew toward general staff also reflects the data collection method, as the questionnaire was distributed primarily through informal and personal networks where employees were more readily accessible and responsive than business owners, who are often occupied with daily operational demands. Importantly, the low representation of IT Staff (10%) itself supports a key argument of this project: most small businesses simply do not employ dedicated technical personnel, reinforcing the need for a security solution that does not depend on specialised in-house expertise.

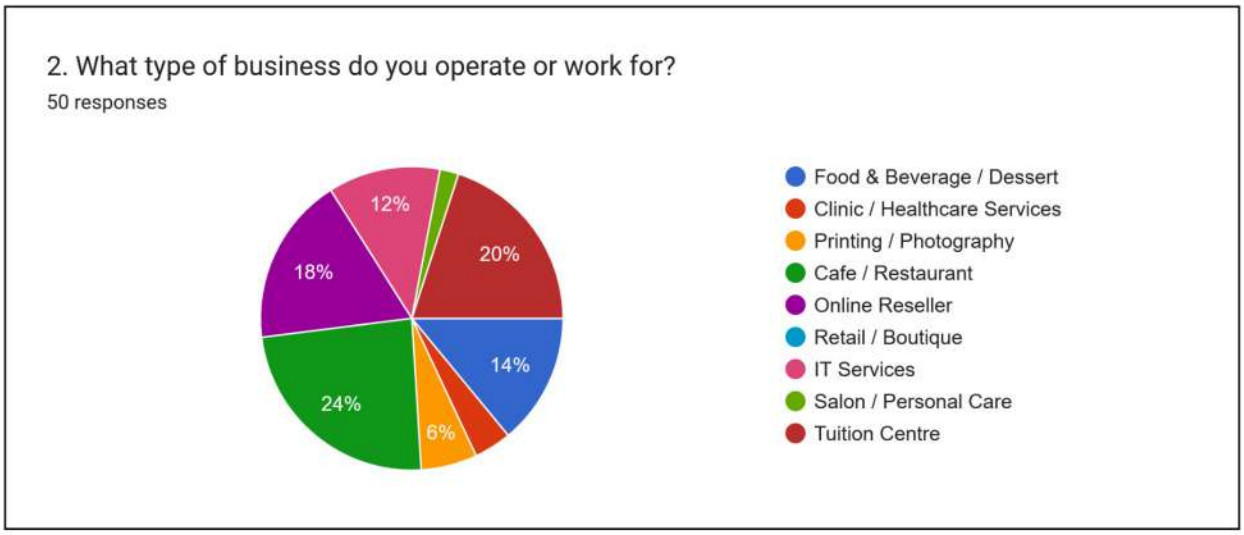


Figure 3.5.1.1.2 *Type of Business*

Respondents represented a diverse range of small business types, with Cafe/Restaurant (24%) and Tuition Centre (20%) being the most common, followed by Online Reseller (18%), Food/Dessert (14%), and IT Services (12%), with smaller representation from Printing, Clinic, and Salon businesses. This variety strengthens the generalisability of the findings, as it demonstrates that the security concerns identified are not isolated to a single industry but are shared broadly across different types of small businesses that rely on networked devices for daily operations. The presence of the client's own business category (Food/Dessert) within the sample also provides a useful point of comparison between the client's specific context and the broader SME population surveyed.

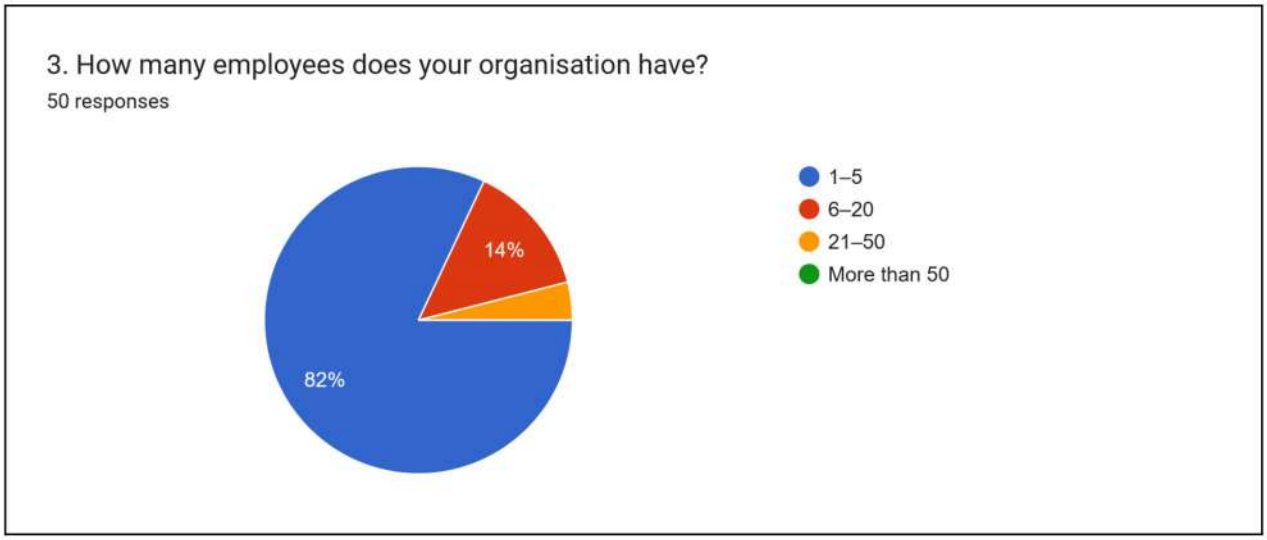


Figure 3.5.1.1.3 *Number of Employees*

The overwhelming majority of respondents, 82% (41 out of 50), operate businesses with only 1 to 5 employees, with a further 14% having 6 to 20 employees, and only 4% having 21 to 50 employees. This confirms that the sample predominantly represents micro-enterprises rather than larger SMEs. This is a significant finding for the project, as it validates that the target demographic genuinely

consists of very small operations with minimal financial and human resources to dedicate toward cybersecurity. Larger SMEs or enterprises are more likely to already possess IT budgets and personnel, making them less representative of the resource-constrained context this project seeks to address.

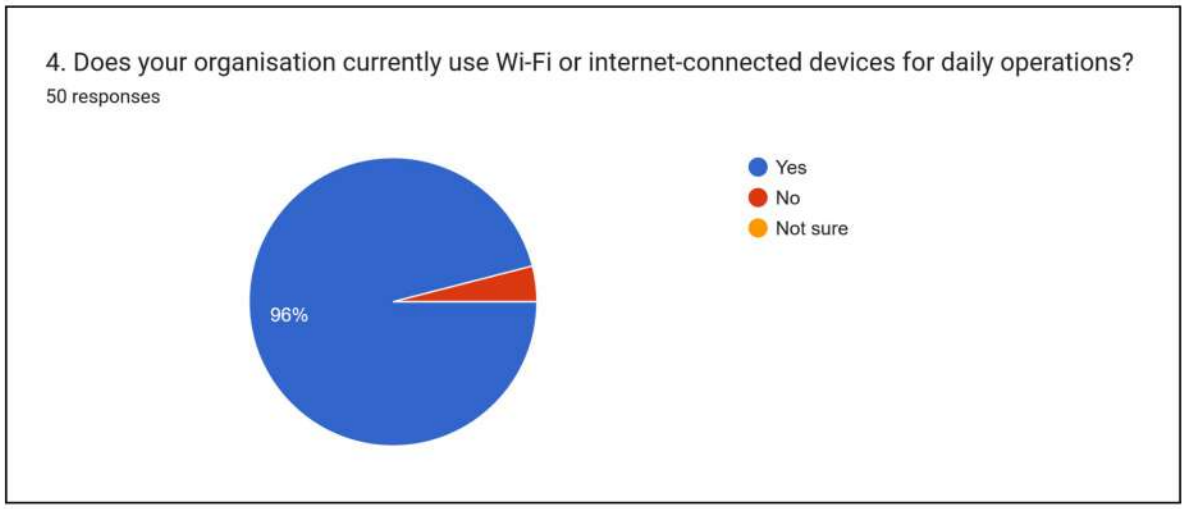


Figure 3.5.1.1.4 *Use of Wi-Fi or Connected Devices*

A near-universal 96% (48 out of 50) of respondents confirmed that their organisation uses Wi-Fi or internet-connected devices for daily operations. This result is unsurprising, as modern business operations, even at the smallest scale, typically depend on point-of-sale systems, online ordering platforms, supplier communication, and digital payment processing, all of which require network connectivity. The two respondents who answered "No" may represent businesses with limited digital operations or may reflect a misunderstanding of the question; nonetheless, the overwhelming majority confirms that network connectivity is now a standard operational requirement even for the smallest businesses, establishing the relevance of network security monitoring regardless of business size.

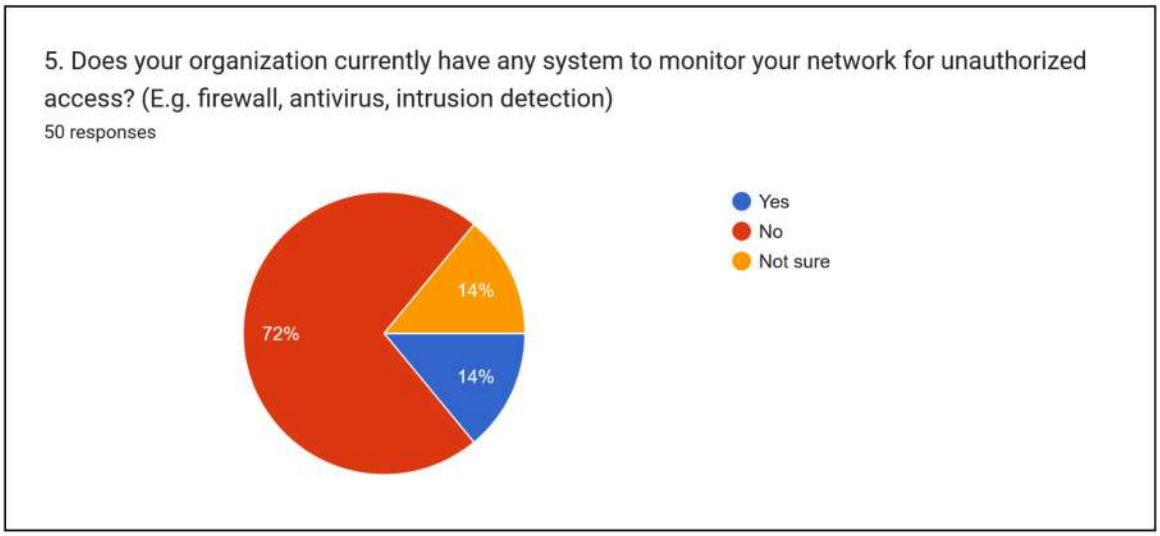


Figure 3.5.1.1.5 *Existence of Network Monitoring Systems*

This question produced one of the most critical findings of the survey: 72% (36 out of 50) of respondents reported having no system to monitor their network for unauthorized access, while a further 14% (7) were unsure, and only 14% (7) confirmed having any monitoring in place. Combining the "No" and "Not sure" responses reveals that 86% of small businesses surveyed either lack network monitoring entirely or are uncertain whether any exists, which for practical purposes amounts to the same level of vulnerability. This finding directly substantiates the first problem statement of this project, namely the lack of network security monitoring among SMEs, and demonstrates that this is not an isolated issue affecting only the client but a widespread condition across the small business sector.

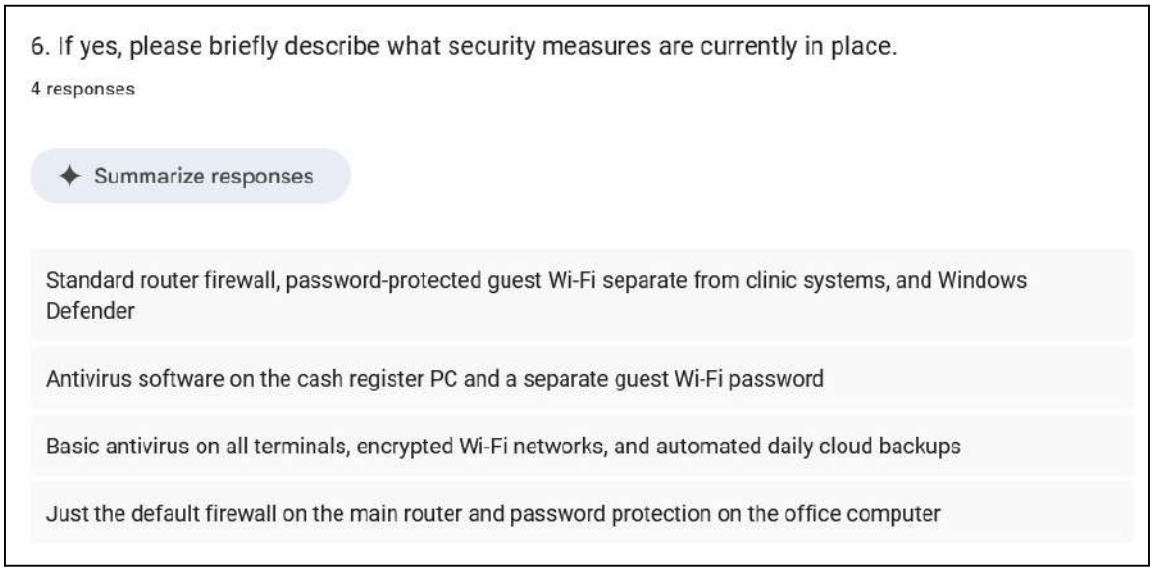


Figure 3.5.1.1.6 *Description of Existing Security Measures*

Among the small number of respondents who elaborated on their existing security measures, the descriptions consistently referred to basic, consumer-grade protections such as default router firewalls, standard antivirus software, password-protected guest Wi-Fi networks, and automated cloud backups. None of the responses described any dedicated intrusion detection, network monitoring, or threat logging capability. This indicates that even among the minority of respondents who believed they had some form of protection in Question 5, what they actually possess falls significantly short of genuine network security monitoring. This gap between perceived and actual protection reinforces the necessity of a dedicated, purpose-built monitoring solution rather than relying on default consumer device settings.

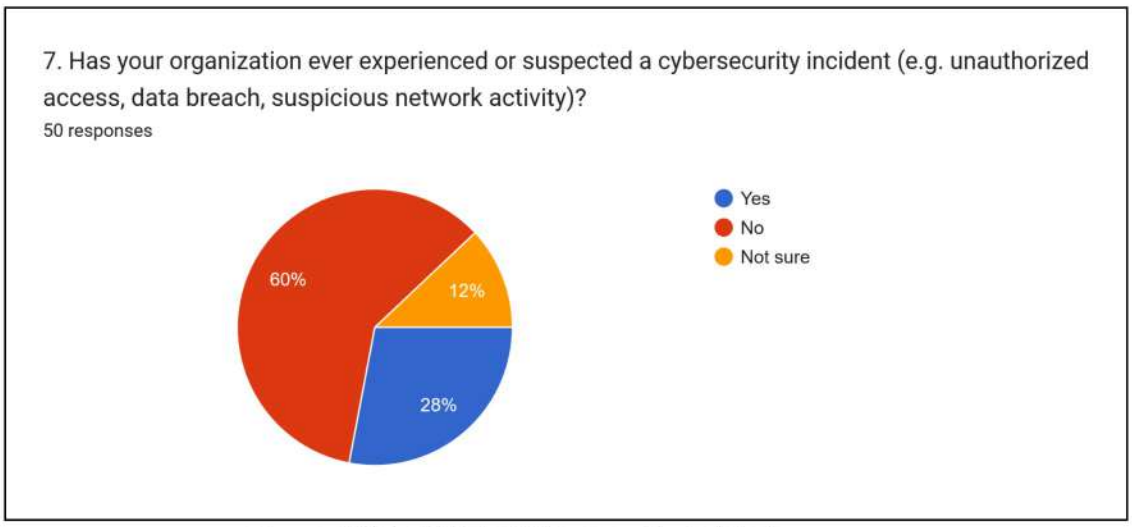


Figure 3.5.1.1.7 *Prior Cybersecurity Incidents*

Despite the high proportion of respondents lacking network monitoring, 28% (14 out of 50) reported having experienced or suspected a cybersecurity incident, while a further 12% (6) were unsure. This is a particularly important finding when considered alongside Question 5: more than a quarter of businesses have already encountered suspicious activity despite the vast majority having no system to detect it. This suggests that the true incidence of unauthorized access attempts on these networks may be considerably higher than reported, as without monitoring tools, many intrusions would likely go entirely unnoticed. The combination of unmonitored networks and confirmed or suspected incidents illustrates a tangible and present risk rather than a purely theoretical concern.

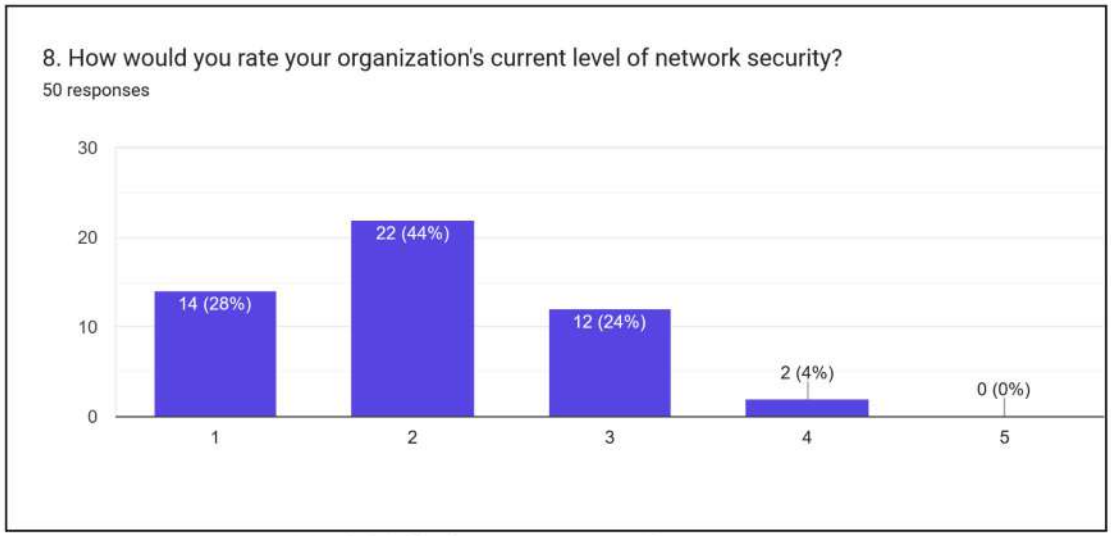


Figure 3.5.1.1.8 *Self-Rated Network Security Level*

Respondents rated their current network security level at an average of 2.04 out of 5, indicating a generally weak self-assessed security posture across the sample. This low rating is consistent with the findings of Question 5, as respondents who openly acknowledged having no monitoring system also rationally rated their security as weak rather than overestimating their protection. This consistency between self-reported practices and self-assessed security level suggests a reasonable

degree of honesty and self-awareness among respondents, strengthening the credibility of the survey findings as a whole.

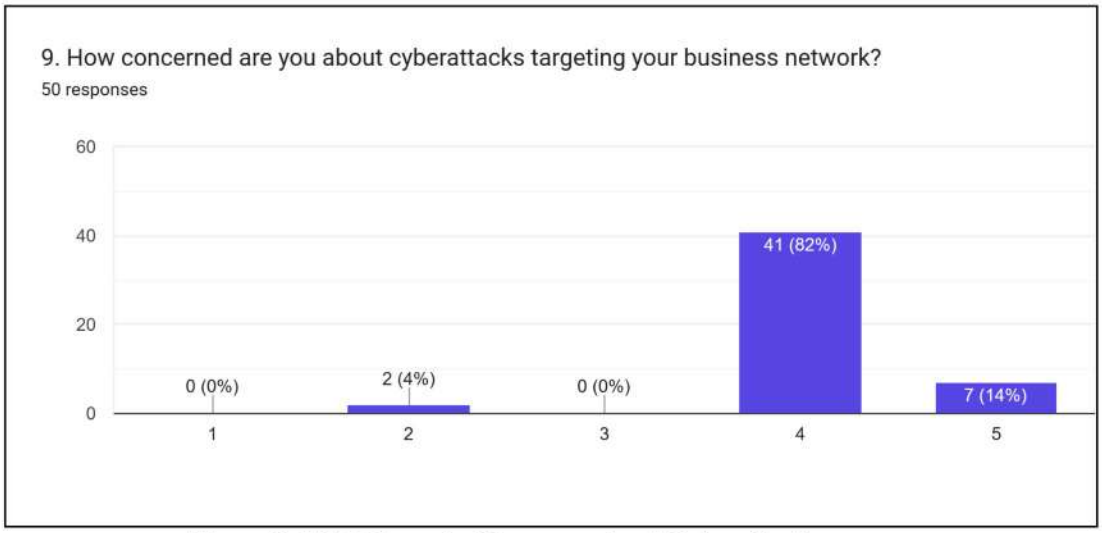


Figure 3.5.1.1.9 *Level of Concern about Cyberattacks*

In contrast to the low security ratings, respondents reported a high average concern level of 4.06 out of 5 regarding cyberattacks targeting their business network. This stark contrast between low actual security (2.04) and high concern (4.06) reveals what may be described as an awareness-action gap: business owners and staff are clearly aware that they are vulnerable and are genuinely worried about this exposure, yet have not taken corresponding action to address it. This gap is a central justification for the proposed system, as it demonstrates that the barrier to improved security is not a lack of awareness or concern, but rather a lack of accessible and affordable solutions, a point further reinforced by the responses to Question 14.

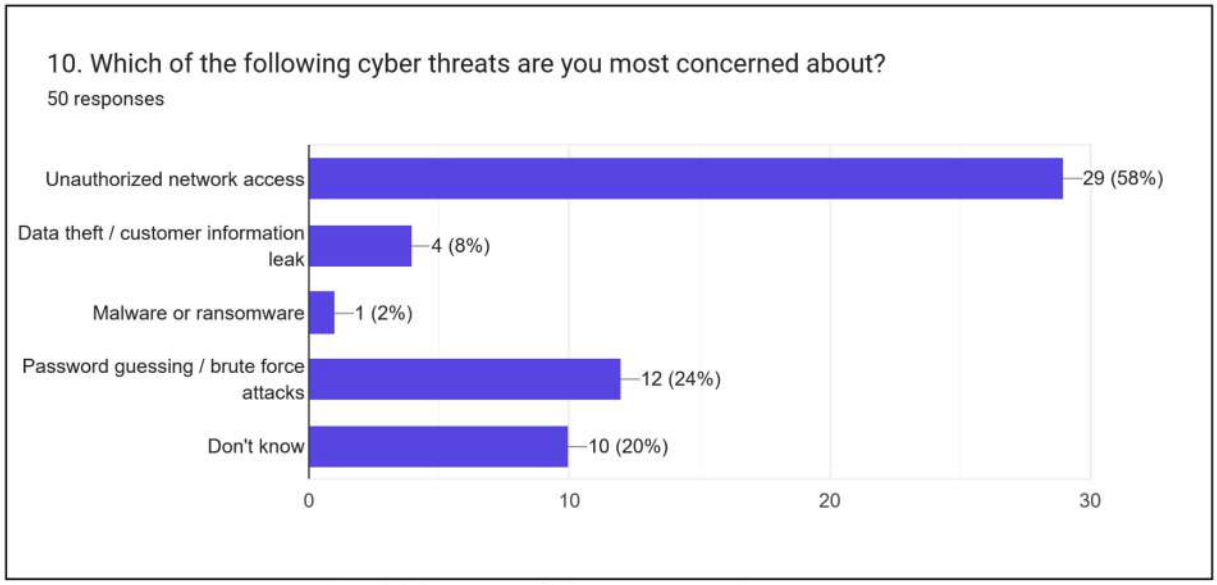


Figure 3.5.1.1.10 *Level of Concern about Cyberattacks*

When asked which cyber threats concerned them most, respondents most frequently selected "Unauthorized network access" (58% of all threat selections), followed by "Password guessing/brute force attacks" (24%), with smaller proportions selecting data theft, malware, or indicating uncertainty. This result is especially significant for this project, as the two most commonly cited concerns, unauthorized access and brute force attempts, are precisely the two primary attack categories that the proposed ESP32 honeypot system is designed to detect. This close alignment between respondent concerns and system capability demonstrates that the proposed solution directly addresses the threats that small business stakeholders themselves identify as most pressing, rather than addressing a hypothetical or assumed need.

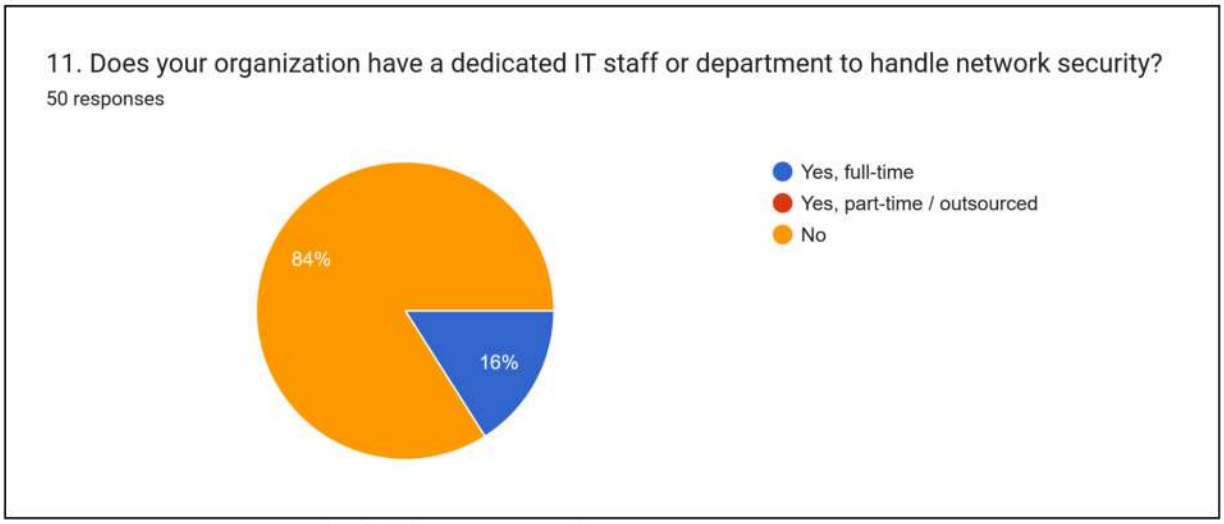


Figure 3.5.1.1.11 *Presence of a Dedicated IT Department*

A substantial 84% (42 out of 50) of respondents indicated that their organisation has no dedicated IT staff or department responsible for network security, with only 16% (8) reporting full-time IT support. This finding is closely related to Question 3, as it confirms that the resource constraints of micro-enterprises extend beyond headcount alone to a specific absence of technical security expertise. This has direct design implications for the proposed system: any solution intended for this demographic must be operable without specialised knowledge, as the typical respondent organisation has no one on staff capable of configuring or interpreting a complex security tool.

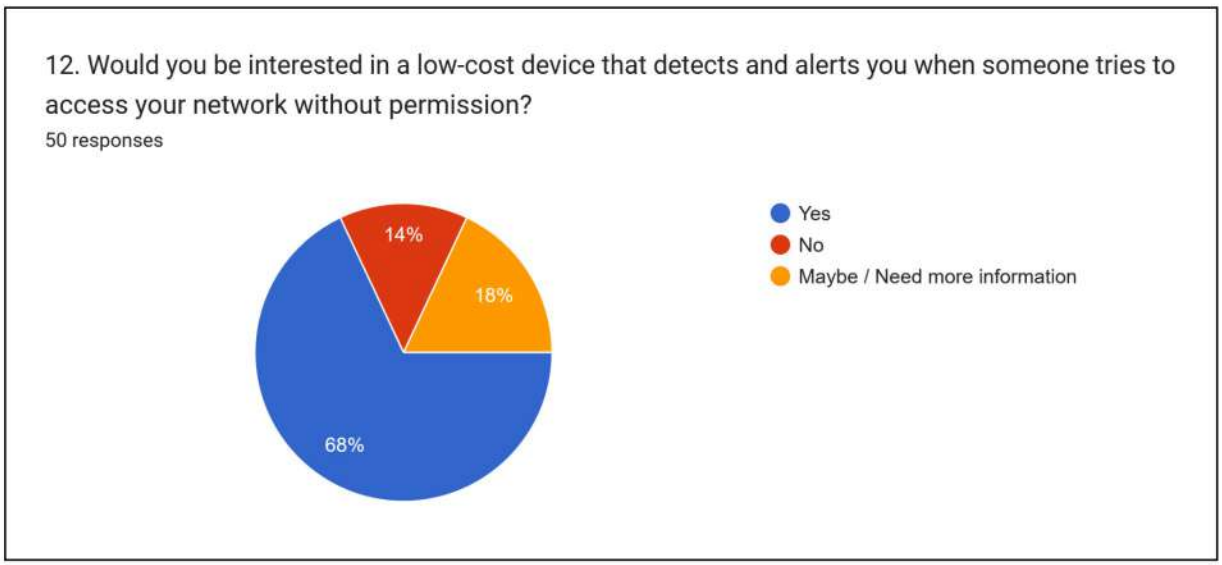


Figure 3.5.1.1.12 *Interest in a Low-Cost Detection Device*

When presented with the concept of a low-cost device capable of detecting and alerting unauthorized network access attempts, 68% (34 out of 50) expressed clear interest, 18% (9) indicated they would need more information, and only 14% (7) were not interested. Closer examination reveals that several of the "No" responses originated from respondents who had previously indicated their organisation already employs full-time IT staff (Question 11), suggesting that their disinterest stems from already feeling adequately protected rather than a rejection of the concept itself. The strong majority interest, particularly among respondents who lack IT support, validates genuine market demand for the proposed solution among its intended target users.

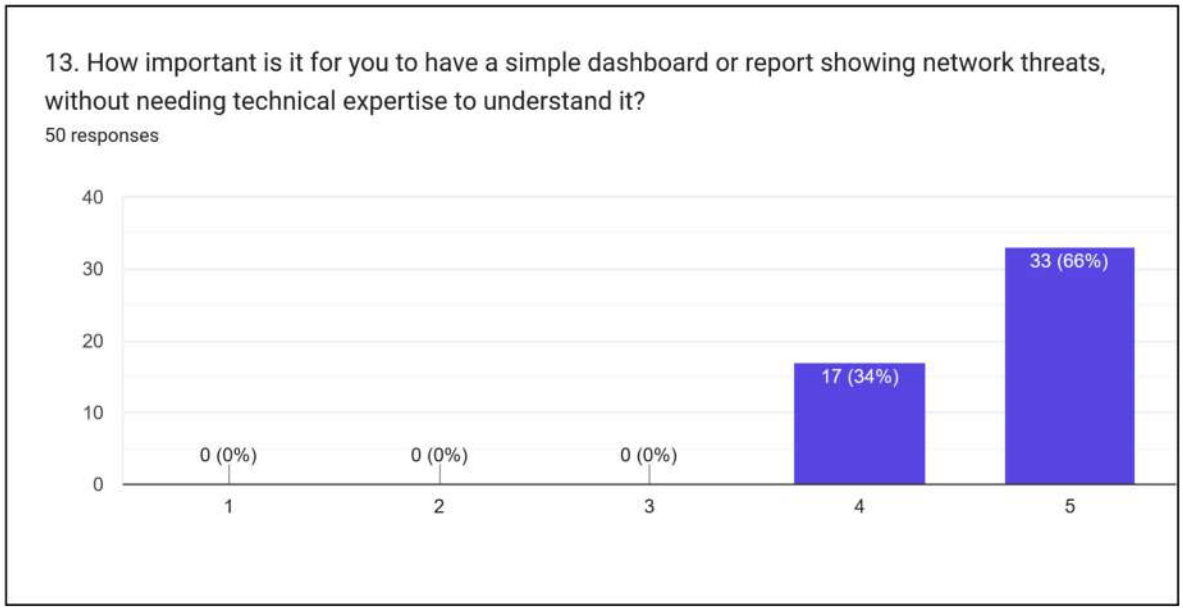


Figure 3.5.1.1.13 *Importance of a Simple Dashboard*

This question received the highest average rating of the entire survey at 4.66 out of 5, indicating that respondents place very high importance on having a simple, non-technical dashboard or report to

understand network threats. This result is best understood in conjunction with Question 11, where 84% of respondents reported having no dedicated IT staff; for these organisations, a complex or technically demanding interface would be effectively unusable, regardless of how capable the underlying detection system might be. This finding has a direct and significant implication for system design, confirming that usability and simplicity of the dashboard are not secondary considerations but a fundamental requirement for the system to deliver genuine value to its target users.

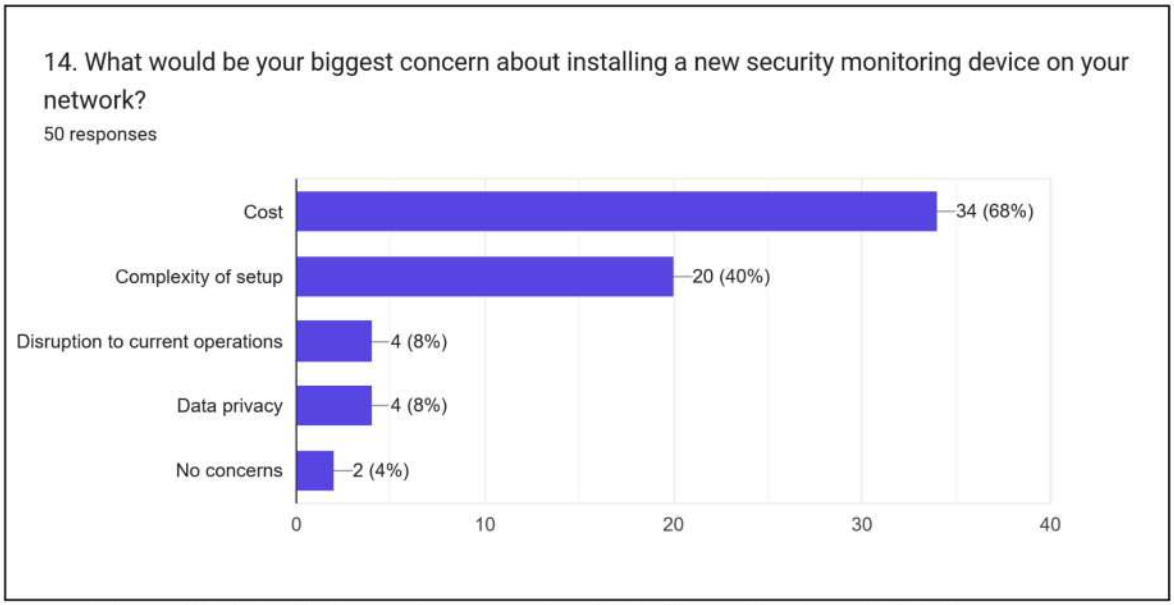


Figure 3.5.1.1.14 *Primary Concerns about Adopting a New Security Device*

Cost emerged as the dominant concern, cited in 68% of all concern selections, followed by complexity of setup at 40%, with disruption to operations and data privacy each receiving smaller but notable mentions. These findings are strongly corroborated by the open-ended comments provided in Question 15, where respondents explicitly stated that "enterprise tools are way too expensive for a small shop like ours" and that any solution "has to be plug-and-play" given the absence of technical staff. This is arguably the most directly applicable finding for this project, as it confirms that the two most significant barriers to security adoption among small businesses, cost and complexity, are precisely the two limitations that the proposed ESP32-based system is specifically designed to overcome through its low hardware cost and simplified dashboard interface.

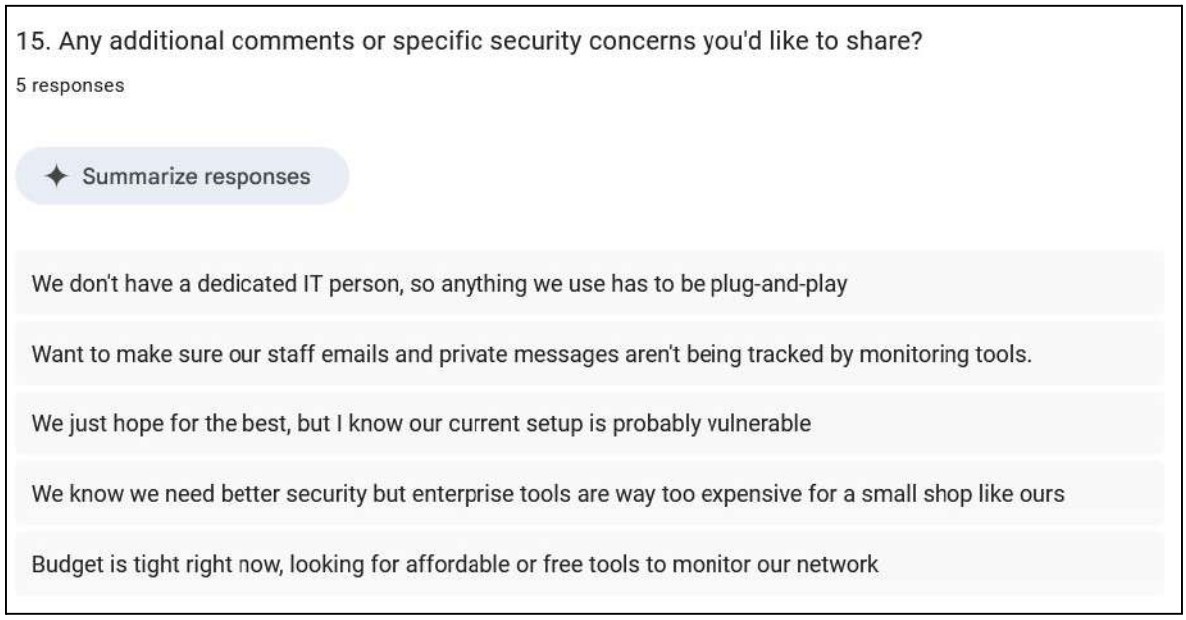


Figure 3.5.1.1.15 *Additional Comments*

The qualitative responses provided in this open-ended question reinforced the quantitative findings throughout the survey. Recurring themes included cost sensitivity, exemplified by comments regarding tight budgets and the high cost of enterprise tools, a desire for plug-and-play simplicity due to the absence of dedicated IT personnel, and a general resigned awareness that current security setups are likely inadequate, as reflected in the comment "we just hope for the best, but I know our current setup is probably vulnerable." One respondent raised a privacy-related concern regarding the possibility of monitoring tools tracking staff communications, which represents a valuable counterpoint that should be acknowledged in the system design, for instance by clarifying that the proposed honeypot monitors unauthorized access attempts rather than legitimate staff activity or communications.

3.5.1.2 Interview Analysis

The interview conducted with Yumplingy Enterprise independently confirmed the findings of the questionnaire within the specific context of the client. The client confirmed that the office WiFi network was installed only a few months prior, purely to support point-of-sale operations and online ordering platforms, and that no firewall, antivirus, or monitoring tool has been put in place since installation. The client further stated that they would not know how to identify suspicious network activity even if it were occurring, and confirmed that no staff member has a technical or information technology background.

When asked about specific concerns, the client identified the protection of customer order and payment data, and the risk of disruption to the point-of-sale system during business hours, as their two primary concerns. The client expressed strong interest in the proposed honeypot system, on the condition that it remains simple to use and does not interfere with existing network performance or daily operations. These findings directly reinforce the non-functional requirements for usability and reliability identified in Section 3.4.3, and confirm that the system design priorities established in this chapter are grounded in the client's actual operating context rather than assumed requirements.

Table 3.5.1.2 *Interview analysis table*

Question	Answer	Analysis
Can you briefly describe Yumplingy Enterprise and its daily operations?	Small dessert/food business handling walk-in and online orders, supplier communication, and payments via POS and online banking.	Confirms a typical micro-SME operational profile — multiple connected systems (POS, banking, online platforms) in daily use, all dependent on the same network
How many staff currently use the office network?	About 4–6 staff connecting phones and laptops daily for order processing, email, and delivery apps.	Small user base, but each device is a potential entry point — confirms the network has real exposure despite the business's small size.
When did you set up your office Wi-Fi, and why?	When did you set up your office Wi-Fi, and why?	Wi-Fi was installed purely for connectivity/business continuity — security was never part of the original decision, directly supporting Problem Statement 1.
Do you have any security measures in place	No, just the standard router from the internet plan. Never really thought about network security.	Confirms zero existing protection — the clearest direct evidence supporting the "lack of network security monitoring" problem statement
Has your business experienced or suspected any unusual network activity?	Not that we've noticed, but we wouldn't know how to check even if something was wrong.	Reveals a critical blind spot — incidents may already be occurring undetected, since the

	Sometimes Wi-Fi feels slower, assumed normal.	client has no way to verify network health or threats.
Do you have IT staff managing your network?	No technical background among any staff. If something goes wrong, we just restart the router or call the ISP.	Confirms zero in-house technical capacity — validates that any solution must be usable without IT expertise (directly shapes Objective 3's dashboard design).
How concerned are you about unauthorized access to your network?	Quite concerned — we store customer orders and payment information, so a breach could be a real problem for us and our customers.	Strong, specific concern tied directly to real business data — validates Problem Statement 1 and 3 with a concrete stake (customer/payment data), not just abstract worry.
What worries you most regarding network security?	Customer data being leaked, or someone interfering with the POS system during business hours — can't afford downtime.	Identifies two specific failure modes (data leak, operational disruption) — reinforces that the solution must be passive/non-intrusive, not just present.
Would a system that detects and alerts unauthorized access attempts be useful?	Yes, definitely — even just knowing something is monitored gives peace of mind. Currently zero visibility.	Direct validation of the project's core value proposition — confirms genuine demand, not assumed need.
How important is simplicity, given no technical staff?	Very important — if it's too complicated or requires reading logs, we probably won't use it properly. Wants a simple dashboard.	Strongest design constraint in the entire interview — directly justifies why Objective 3's dashboard must be non-technical, echoing the questionnaire's Q13 finding (4.66/5 importance).
Comfortable connecting a monitoring device to the office Wi-Fi?	Yes, as long as it doesn't slow down internet or interfere with the POS system.	Confirms acceptance is conditional on zero operational disruption — validates the project's low-interaction, passive-monitoring design choice
Any specific features wanted in the dashboard?	Just show when an attack happens, what type, and from where if possible. Nothing too detailed.	Client's own words almost exactly match the system's actual output (attack type, IP, timestamp) — strong evidence the system was designed around real stated needs, not assumptions.

3.5.2 Use Case Model

Figure 3.1 presents the use case diagram of the proposed system, identifying three actors and their respective interactions with the system. The Attacker actor represents any unauthorized party that connects to the decoy services exposed by the ESP32 honeypot, and may perform a port scan, attempt a brute-force login through either the Telnet or HTTP service, or make an unauthorized connection attempt. The Staff actor represents the management and staff of Yumplingy Enterprise, who view the live dashboard, which includes the alert banner, and may generate a forensic report. The Admin actor represents the party responsible for managing the system's database, who may manage records through the Django administrative panel and review all stored attack logs.

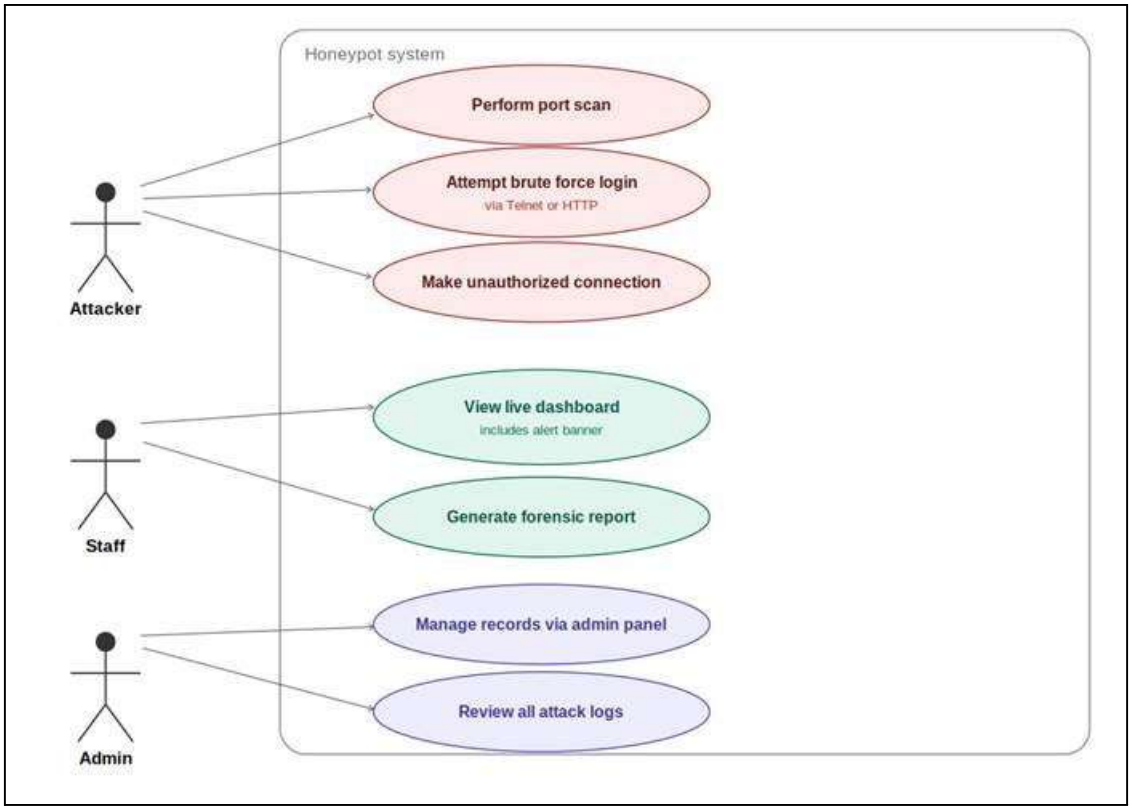


Figure 3.5.2 *Use case diagram of the proposed ESP32-based IoT honeypot system*

3.5.3 Flowchart

Figure 3.2 presents the process flowchart of the proposed system, tracing the sequence of events from the moment a connection is received by the ESP32 to the point at which the resulting record becomes available to Staff and Admin. Upon receiving a connection on either port 23 or port 80, the system first checks whether the same source IP address has connected repeatedly within a short time window, classifying the event as a port scan if this threshold is met. If not, the system routes the connection to either the fake Telnet service or the fake HTTP service, depending on the port accessed, before classifying the resulting interaction as a brute-force attempt or a simple connection attempt. The classified event is then transmitted as a JSON payload over HTTP POST to the Django backend, which processes and stores the record in the MySQL database, after which the dashboard is updated with an alert for Staff and the record becomes available for review by Admin.

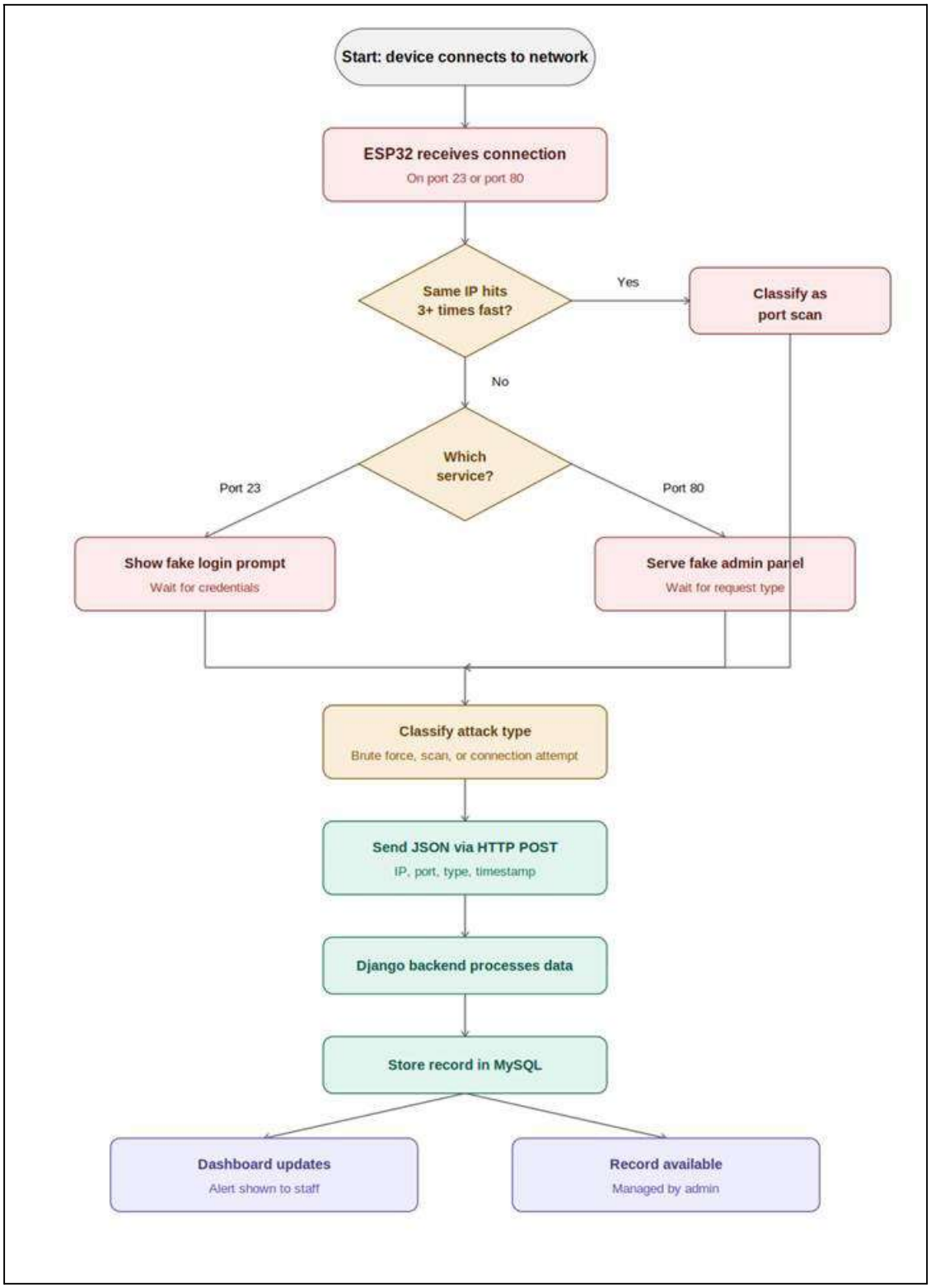


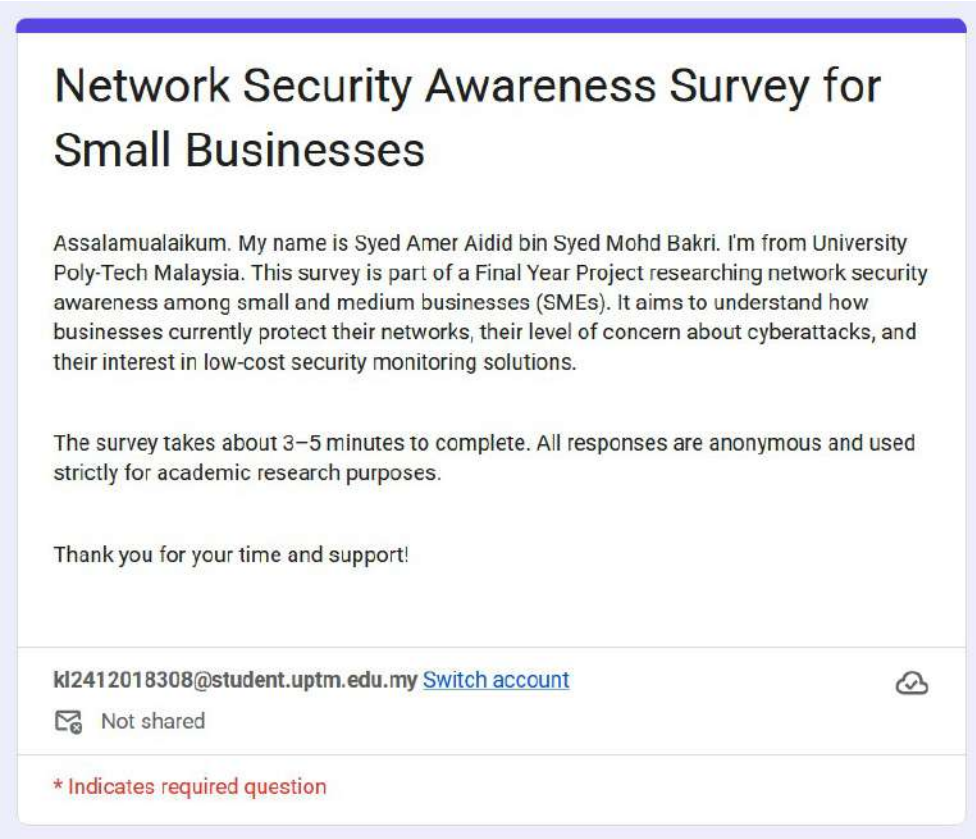
Figure 3.5.3 *Process flowchart of the proposed honeypot detection pipeline*

3.6 Conclusion

This chapter has presented the methodology adopted for the development of the proposed ESP32-Based IoT Honeypot System. The Agile methodology was justified as the most suitable approach given the need to refine the system iteratively based on feedback from a non-technical client, and was structured into four phases covering planning, design, development, and testing. The requirements of the system were then established through two data gathering techniques, a questionnaire distributed to a general sample of small business respondents and a structured interview conducted with Yumplingy Enterprise, from which functional, non-functional, and system requirements were derived. Finally, the analysis of the questionnaire and interview findings was presented alongside the use case model, process flowchart, and BPMN diagram, which together model the proposed system's behaviour and the responsibilities of its three actors, namely the Attacker, Staff, and Admin. These findings and models provide the foundation for the system design and implementation work to be carried out in the remainder of this project.

Appendix A – Requirements Specification Document

- Questionnaire



Network Security Awareness Survey for Small Businesses

Assalamualaikum. My name is Syed Amer Aidid bin Syed Mohd Bakri. I'm from University Poly-Tech Malaysia. This survey is part of a Final Year Project researching network security awareness among small and medium businesses (SMEs). It aims to understand how businesses currently protect their networks, their level of concern about cyberattacks, and their interest in low-cost security monitoring solutions.

The survey takes about 3–5 minutes to complete. All responses are anonymous and used strictly for academic research purposes.

Thank you for your time and support!

kl2412018308@student.uptm.edu.my Switch account

Not shared

* Indicates required question

1. What is your role in the organisation? *

- ◯ Business Owner / Management
- ◯ IT Staff
- ◯ General Staff / Employee
- ◯ Other: _____

2. What type of business do you operate or work for? *

- ◯ Food & Beverage / Dessert
- ◯ Clinic / Healthcare Services
- ◯ Printing / Photography
- ◯ Cafe / Restaurant
- ◯ Online Reseller
- ◯ Retail / Boutique
- ◯ IT Services
- ◯ Salon / Personal Care
- ◯ Tuition Centre

3. How many employees does your organisation have? *

- ◯ 1–5
- ◯ 6–20
- ◯ 21–50
- ◯ More than 50

4. Does your organisation currently use Wi-Fi or internet-connected devices for daily operations? *

- ◯ Yes
- ◯ No
- ◯ Not sure

5. Does your organization currently have any system to monitor your network for unauthorized access? (E.g. firewall, antivirus, intrusion detection) *

- ◯ Yes
- ◯ No
- ◯ Not sure

6. If yes, please briefly describe what security measures are currently in place.

Your answer

7. Has your organization ever experienced or suspected a cybersecurity incident (e.g. unauthorized access, data breach, suspicious network activity)? *

- ◯ Yes
- ◯ No
- ◯ Not sure

8. How would you rate your organization's current level of network security? *

	1	2	3	4	5	
Very Weak	◯	◯	◯	◯	◯	Very Strong

9. How concerned are you about cyberattacks targeting your business network? *

	1	2	3	4	5	
Not Concerned	○	○	○	○	○	Extremely Concerned

10. Which of the following cyber threats are you most concerned about? *

- [ ] Unauthorized network access
- [ ] Data theft / customer information leak
- [ ] Malware or ransomware
- [ ] Password guessing / brute force attacks
- [ ] Don't know

11. Does your organization have a dedicated IT staff or department to handle network security? *

- ○ Yes, full-time
- ○ Yes, part-time / outsourced
- ○ No

12. Would you be interested in a low-cost device that detects and alerts you when someone tries to access your network without permission? *

- ○ Yes
- ○ No
- ○ Maybe / Need more information

13. How important is it for you to have a simple dashboard or report showing network threats, without needing technical expertise to understand it? *

	1	2	3	4	5	
Not Important	○	○	○	○	○	Very Important

14. What would be your biggest concern about installing a new security monitoring device on your network? *

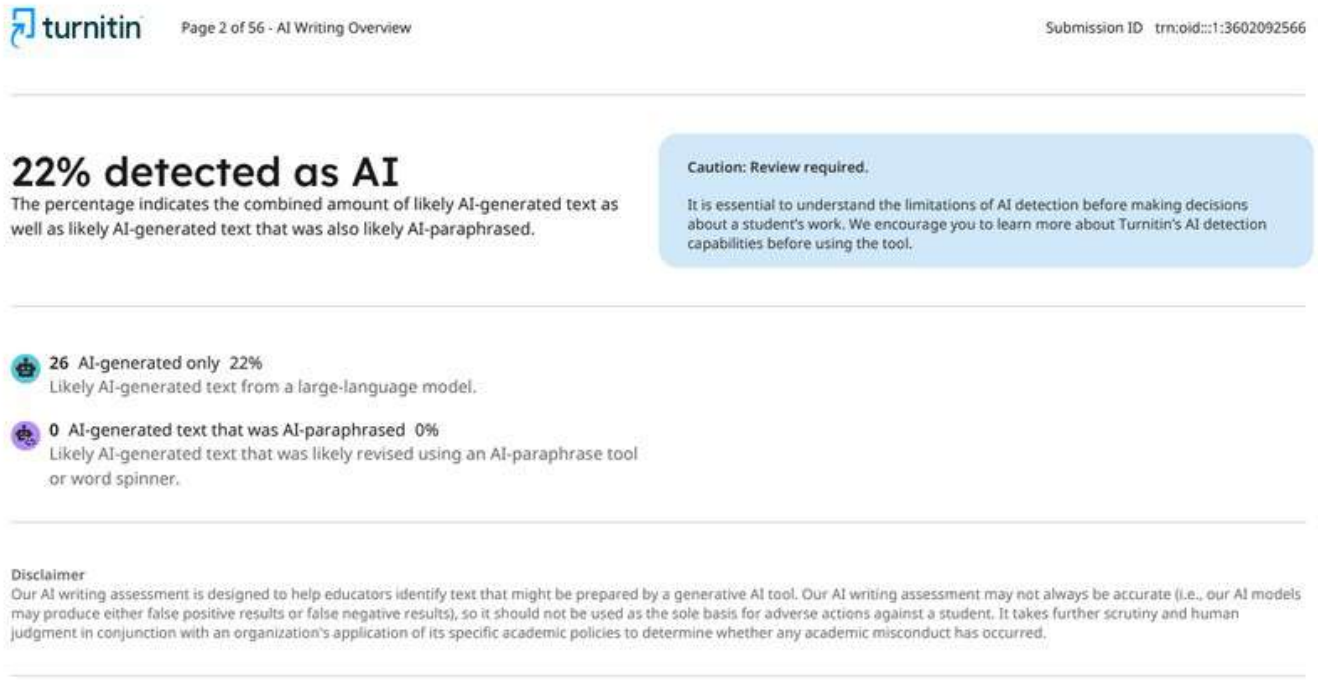
- [ ] Cost
- [ ] Complexity of setup
- [ ] Disruption to current operations
- [ ] Data privacy
- [ ] No concerns
- [ ] Other: _____

15. Any additional comments or specific security concerns you'd like to share?

Your answer

Submit Clear form

Appendix B – Turnitin Result



turnitin Page 2 of 56 - AI Writing Overview Submission ID trn:oid:::1:3602092566

22% detected as AI

The percentage indicates the combined amount of likely AI-generated text as well as likely AI-generated text that was also likely AI-paraphrased.

Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

26 AI-generated only 22%
Likely AI-generated text from a large-language model.

0 AI-generated text that was AI-paraphrased 0%
Likely AI-generated text that was likely revised using an AI-paraphrase tool or word spinner.

Disclaimer
Our AI writing assessment is designed to help educators identify text that might be prepared by a generative AI tool. Our AI writing assessment may not always be accurate (i.e., our AI models may produce either false positive results or false negative results), so it should not be used as the sole basis for adverse actions against a student. It takes further scrutiny and human judgment in conjunction with an organization's application of its specific academic policies to determine whether any academic misconduct has occurred.

FYP Report (2)_compressed.pdf

ORIGINALITY REPORT

8%	5%	4%	6%
SIMILARITY INDEX	INTERNET SOURCES	PUBLICATIONS	STUDENT PAPERS

PRIMARY SOURCES

1	www.mdpi.com	Internet Source
2	Submitted to Universiti Poly-Tech Malaysia	Student Paper
3	Submitted to Campbellsville University	Student Paper
4	Submitted to Canterbury Institute of Management	Student Paper
5	files.sdiarticle5.com	Internet Source
6	sites.wp.odu.edu	Internet Source
7	eprints.uptm.edu.my	Internet Source
8	Edivaldo Pastori Valentini, Thiago Araujo Franco, Euclydes Nasorri Gottsfritz, Julio Cezar Estrella et al. "Honeypot Embedded in Low-Cost IoT Hardware for Collecting and Analyz Real-World Cyber Threats", Proceedings of the 18th IEEE/ACM International Conference on Utility and Cloud Computing, 2025	Publication
9	Submitted to Middle East College	Student Paper
10	glisc.info	Internet Source
11	Degefu, Mesay Workneh. "Challenges Microbusinesses Face in Managing Cyber Risk: A Qualitative Analysis Exploring Cyber Insurance as a Response to Cyber Risk.", Marymou University	Publication
12	iieta.org	Internet Source
13	Submitted to Middlesex University	Student Paper

14 Submitted to University of Waikato
Student Paper

15 Submitted to University of Greenwich
Student Paper

16 Submitted to University College Technology Sarawak
Student Paper

17 Submitted to Buckinghamshire Chilterns University College
Student Paper

18 Submitted to University of Hertfordshire
Student Paper

19 1library.net
Internet Source

20 Submitted to PSB Academy
Student Paper

21 Submitted to University of Technology, Sydney
Student Paper

22 aiprimelab.com
Internet Source

23 repository.sustech.edu
Internet Source

24 Submitted to universititeknologimara
Student Paper

25 Submitted to American Intercontinental University Online
Student Paper

26 Submitted to Asia Pacific Instutute of Information Technology
Student Paper

27 Shravan Pargaonkar. "A Guide to Software Quality Engineering", CRC Press, 2024
Publication

28 etheses.dur.ac.uk
Internet Source

29 He, Yitong. "Network Security Management Methods for Smes.", University of Johannesburg (South Africa), 2024
Publication

30 vdocument.in
Internet Source

31 Dinesh Goyal, Shabina Jameer Modi, Anil Kumar, Vinit Khetani. "Advancement in Engineering and Science - Recent Trends and Applications", CRC Press, 2026
Publication

32 Submitted to Technological University Dublin
Student Paper

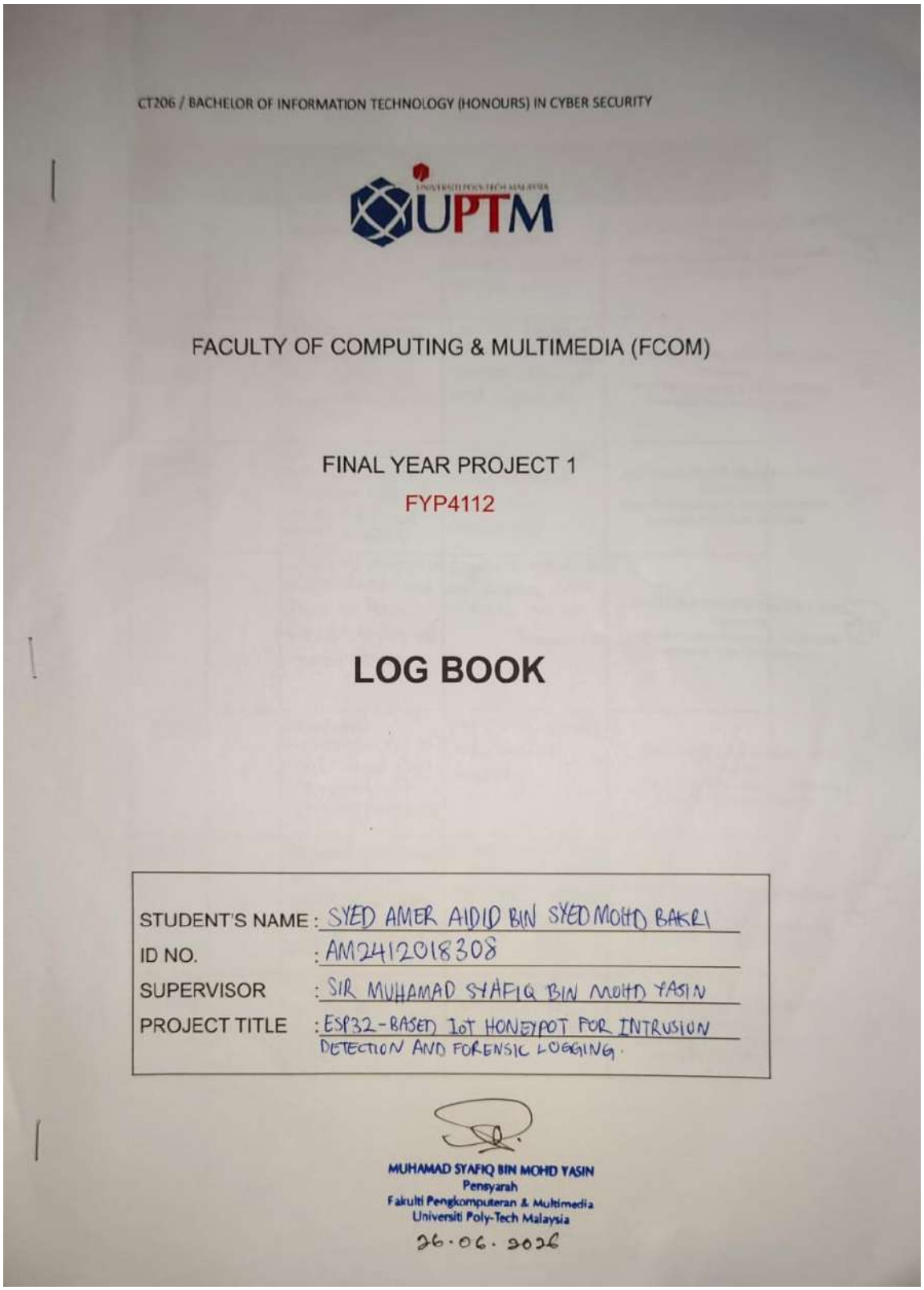
33 core.ac.uk
Internet Source

34 www.ijircce.com
Internet Source

35 Amir Javadpour, Forough Ja'fari, Tarik Taleb, Mohammad Shojafar, Chafika Benzaïd. "A Comprehensive Survey on Cyber Deception Techniques to Improve Honeypot Performance", Computers & Security, 2024
Publication

Exclude quotes Off
Exclude bibliography Off
Exclude matches Off

Appendix C – Log Book



CT206 / BACHELOR OF INFORMATION TECHNOLOGY (HONOURS) IN CYBER SECURITY

UPTM

FACULTY OF COMPUTING & MULTIMEDIA (FCOM)

FINAL YEAR PROJECT 1

FYP4112

LOG BOOK

STUDENT'S NAME : SYED AMER AIDID BIN SYED MOHD BAKRI
ID NO. : AM2412018308
SUPERVISOR : SIR MUHAMAD SYAFIQ BIN MOHD YASIN
PROJECT TITLE : ESP32-BASED IoT HONEYPOT FOR INTRUSION DETECTION AND FORENSIC LOGGING.

MUHAMAD SYAFIQ BIN MOHD YASIN
Pensyarah
Fakulti Pengkomputeran & Multimedia
Universiti Poly-Tech Malaysia
26.06.2026

CT206 / BACHELOR OF INFORMATION TECHNOLOGY (HONOURS) IN CYBER SECURITY

Week		Agenda	Next Agenda	Signature (Supervisor / Coordinator)
13/5/26	1	Brainstorming, Introduction to the course and Title selection	Begin research on honeypot concepts and find a project client	MUHAMAD SYAFIQ BIN MOHD YASIN Pensyarah Fakulti Pengkomputeran & Multimedia Universiti Poly-Tech Malaysia
~~21/5/26~~ 20/5/26	2	Confirmed Yumpling Enterprise as client and drafted the Project Brief Form	Continue literature review and finalize problem statement and objectives	MUHAMAD SYAFIQ BIN MOHD YASIN Pensyarah Fakulti Pengkomputeran & Multimedia Universiti Poly-Tech Malaysia
~~25/5/26~~ 29/5/26	3	conducted literature review on honeypot framework (Honeyd, cowrie, T-Pot) and related academic papers	Develop a comparison of related works and identify research gap	MUHAMAD SYAFIQ BIN MOHD YASIN Pensyarah Fakulti Pengkomputeran & Multimedia Universiti Poly-Tech Malaysia
3/6/26	4	completed comparison of related works and discussion linking findings to proposed system design	Distribute questionnaire and prepare client interview questions	MUHAMAD SYAFIQ BIN MOHD YASIN Pensyarah Fakulti Pengkomputeran & Multimedia Universiti Poly-Tech Malaysia
10/6/26	5	Distributed questionnaire (n=50) and conducted client interview ~~ed~~ with Yumpling Enterprise	Analyse questionnaire and interview findings.	MUHAMAD SYAFIQ BIN MOHD YASIN Pensyarah Fakulti Pengkomputeran & Multimedia Universiti Poly-Tech Malaysia
15/6/26	6	Analysed questionnaire and interview results, and began writing Chapter 1 until Chapter 3	Continue writing report and design use case diagram and flow-chart	MUHAMAD SYAFIQ BIN MOHD YASIN Pensyarah Fakulti Pengkomputeran & Multimedia Universiti Poly-Tech Malaysia
24/6/26	7	Completed FYP1 report and prepared presentation slide	Finalise and rehearse FYP1 Presentation	MUHAMAD SYAFIQ BIN MOHD YASIN Pensyarah Fakulti Pengkomputeran & Multimedia Universiti Poly-Tech Malaysia

References

Bellekens, X. (2022, June 13). *Set up T-POT honeypot in the cloud in less than 30 minutes*. Lupovis. https://www.lupovis.io/set-up-t-pot-honeypot-in-cloud-in-less-than-30-minutes/

Chidukwani, A., Zander, S., & Koutsakis, P. (2022). A survey on the cyber security of small-to-medium businesses: Challenges, research focus and recommendations. *IEEE Access*, *10*(10), 85701–85719. https://doi.org/10.1109/access.2022.3197899

French, D. (2018, October 1). *How to Setup "Cowrie" — An SSH Honeypot*. Medium. https://medium.com/threatpunter/how-to-setup-cowrie-an-ssh-honeypot-535a68832e4c

Gajjar, B. (2013, October 12). *Common problems during initial Honeyd configuration*. Bhumish Gajjar's Blog. https://bhumishgajjar.wordpress.com/2013/10/12/common-problems-with-initial-honeyd-configuration/

GeeksforGeeks. (2023, December 13). *What is Agile Methodology?* GeeksforGeeks. https://www.geeksforgeeks.org/software-testing/what-is-agile-methodology/

Javadpour, A., Ja'fari, F., Taleb, T., Shojafar, M., & Benzaïd, C. (2024). A Comprehensive Survey on Cyber Deception Techniques to Improve Honeypot Performance. *Computers & Security*, *140*. https://doi.org/10.1016/j.cose.2024.103792

Mohd Fuzi, M. F., & Mazlan, M. F. (2024). Performance analysis of network intrusion detection using T-Pot honeypots. *Journal of Computing Research and Innovation*, *9*(2), 348–360. https://doi.org/10.24191/jcrinn.v9i2.477

Moric, Z., Dakic, V., & Regvart, D. (2025). Advancing Cybersecurity with Honeypots and Deception Strategies. *Informatics*, *12*(1), 14–14. https://doi.org/10.3390/informatics12010014

Negi, P. S., Garg, A., & Lal, R. (2020, January 1). *Intrusion Detection and Prevention using Honeypot Network for Cloud Security*. IEEE Xplore. https://doi.org/10.1109/Confluence47617.2020.9057961

OWASP Internet of Things | OWASP Foundation. (2018). Owasp.org. https://owasp.org/www-project-internet-of-things

Provos, N. (2004). *A Virtual Honeypot Framework*. Www.usenix.org. https://www.usenix.org/conference/13th-usenix-security-symposium/virtual-honeypot-framework

Technical Documents | Espressif Systems. (n.d.). Www.espressif.com. https://docs.espressif.com

Valentini, E. P., Franco, T. A., Gottsfritz, E. N., Estrella, J. C., Júnior, L. A. P., & Meneguette, R. I. (2025). Honeypot Embedded in Low-Cost IoT Hardware for Collecting and Analyzing Real-World Cyber Threats. *Proceedings of the 18th IEEE/ACM International Conference on Utility and Cloud Computing* , 1–6. https://doi.org/10.1145/3773274.3774930

Verizon. (2025). *2025 Data Breach Investigations Report*. Verizon Business. https://www.verizon.com/business/resources/reports/dbir/

Wang, S., Tu, C., Zhang, Y., Zhang, M., & Xue, P. (2025). Mapping Cyber Bot Behaviors: Understanding Payload Patterns in Honeypot Traffic. *Sensors*, *26*(1), 11. https://doi.org/10.3390/s26010011

Zielinski, D., & Kholidy, Hisham A. (2022). *An Analysis of Honeypots and their Impact as a Cyber Deception Tactic*. ArXiv.org. https://arxiv.org/abs/2301.00045